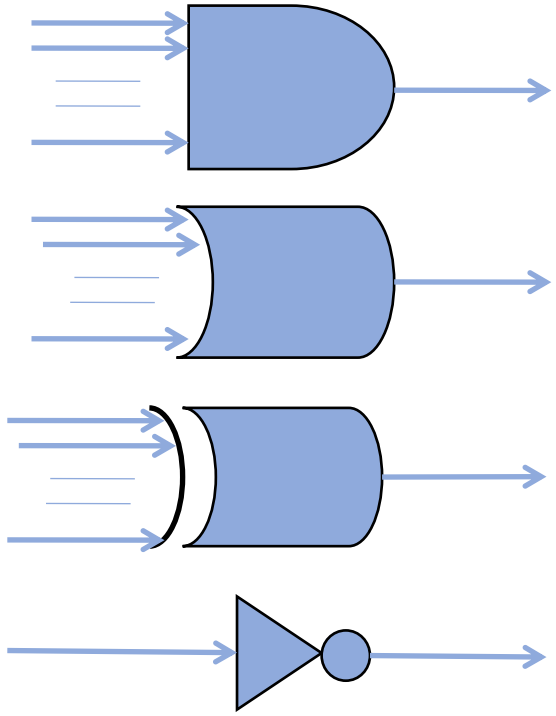
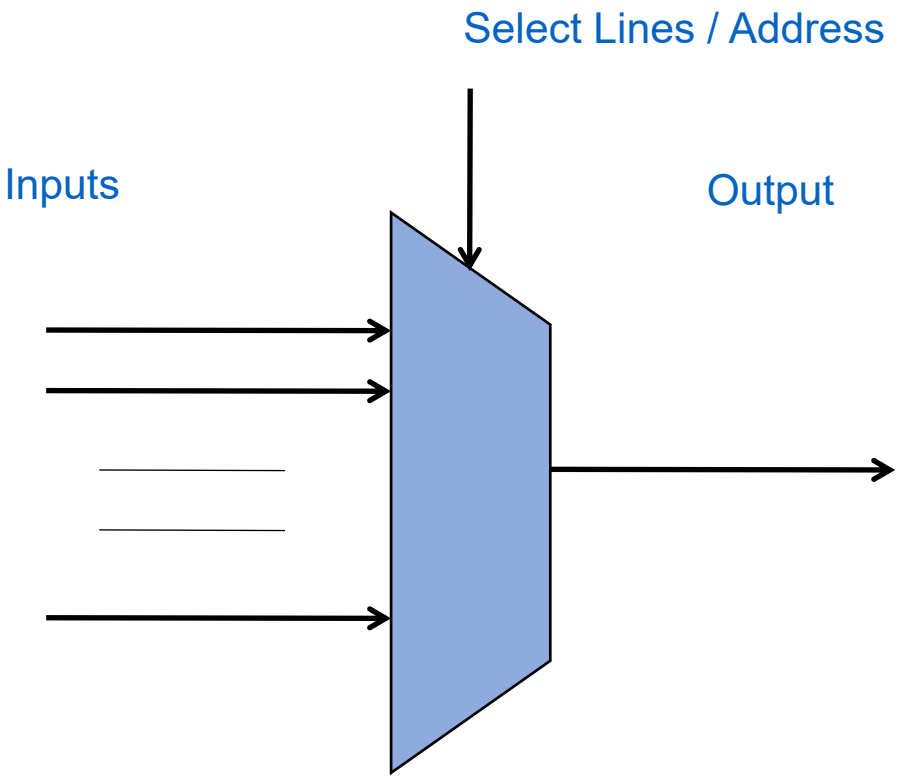
Building Blocks

Gates

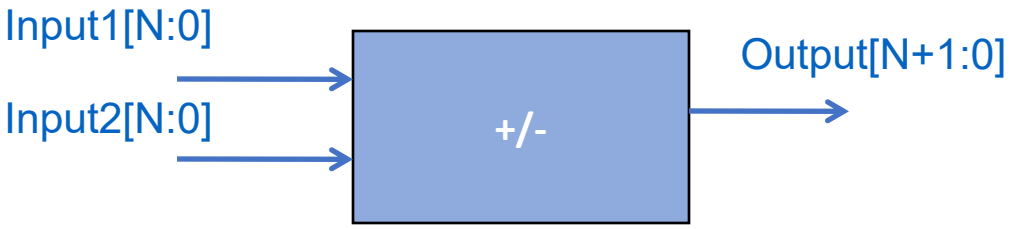
Inputs Outputs



Muxes and Demuxes

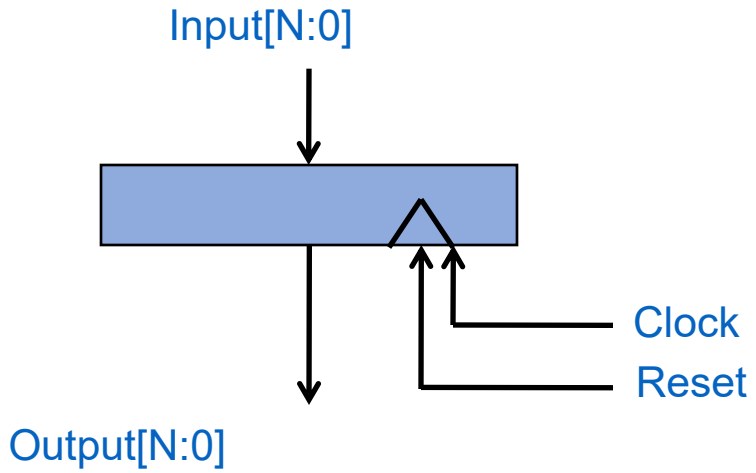


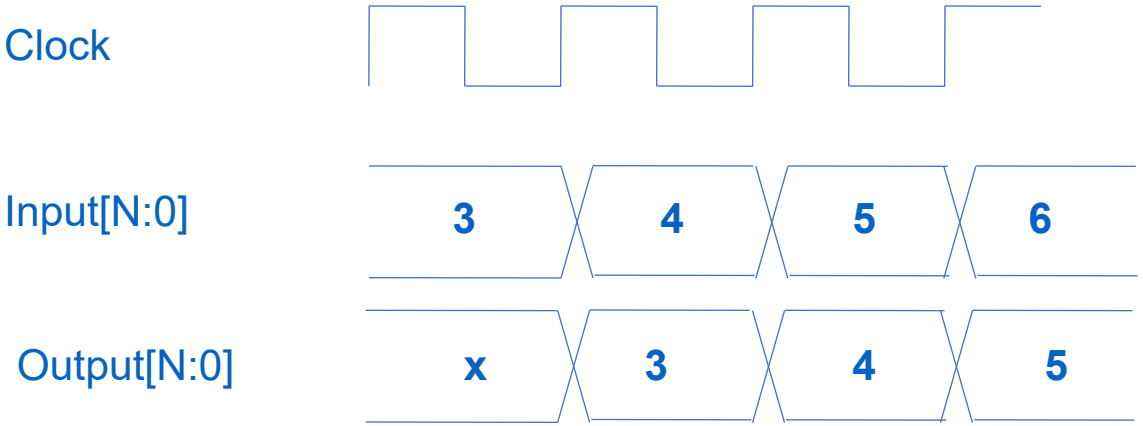
Adders and Multipliers

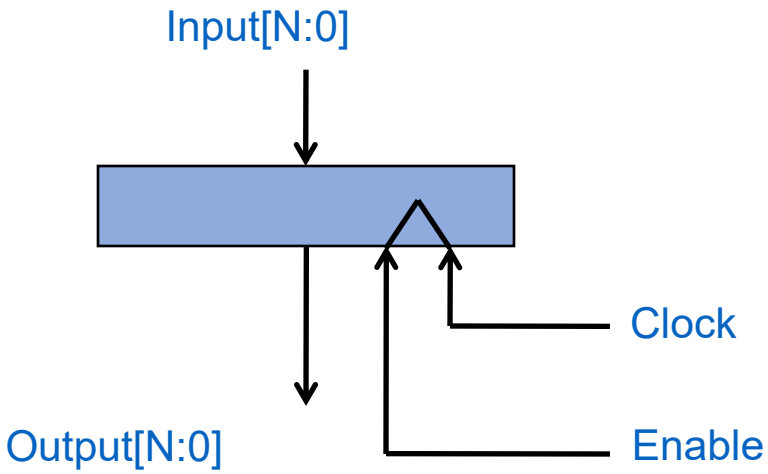


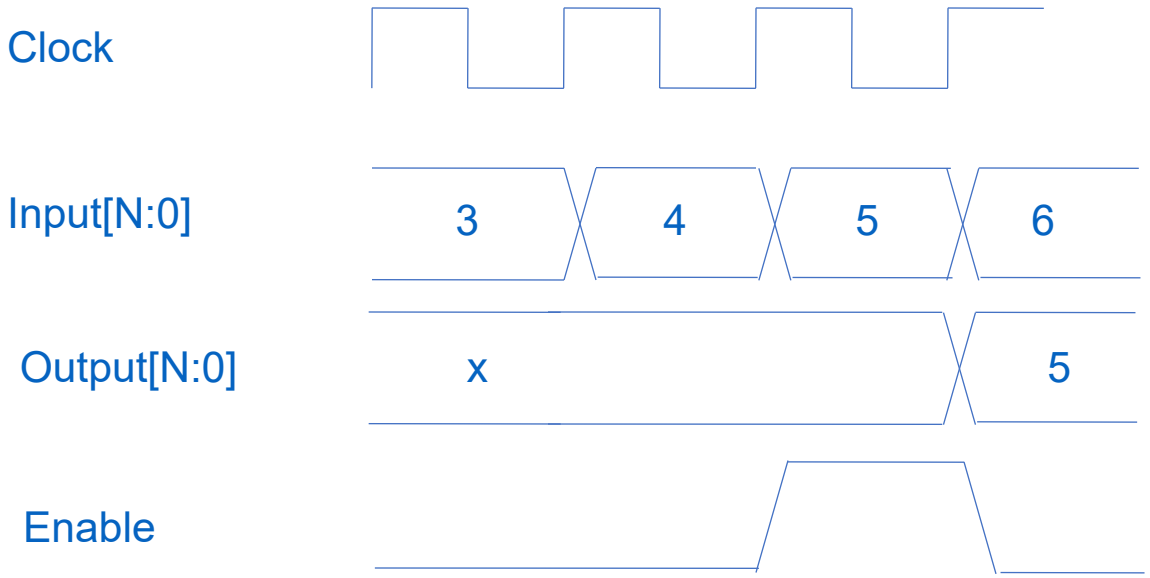


Registers/Flops

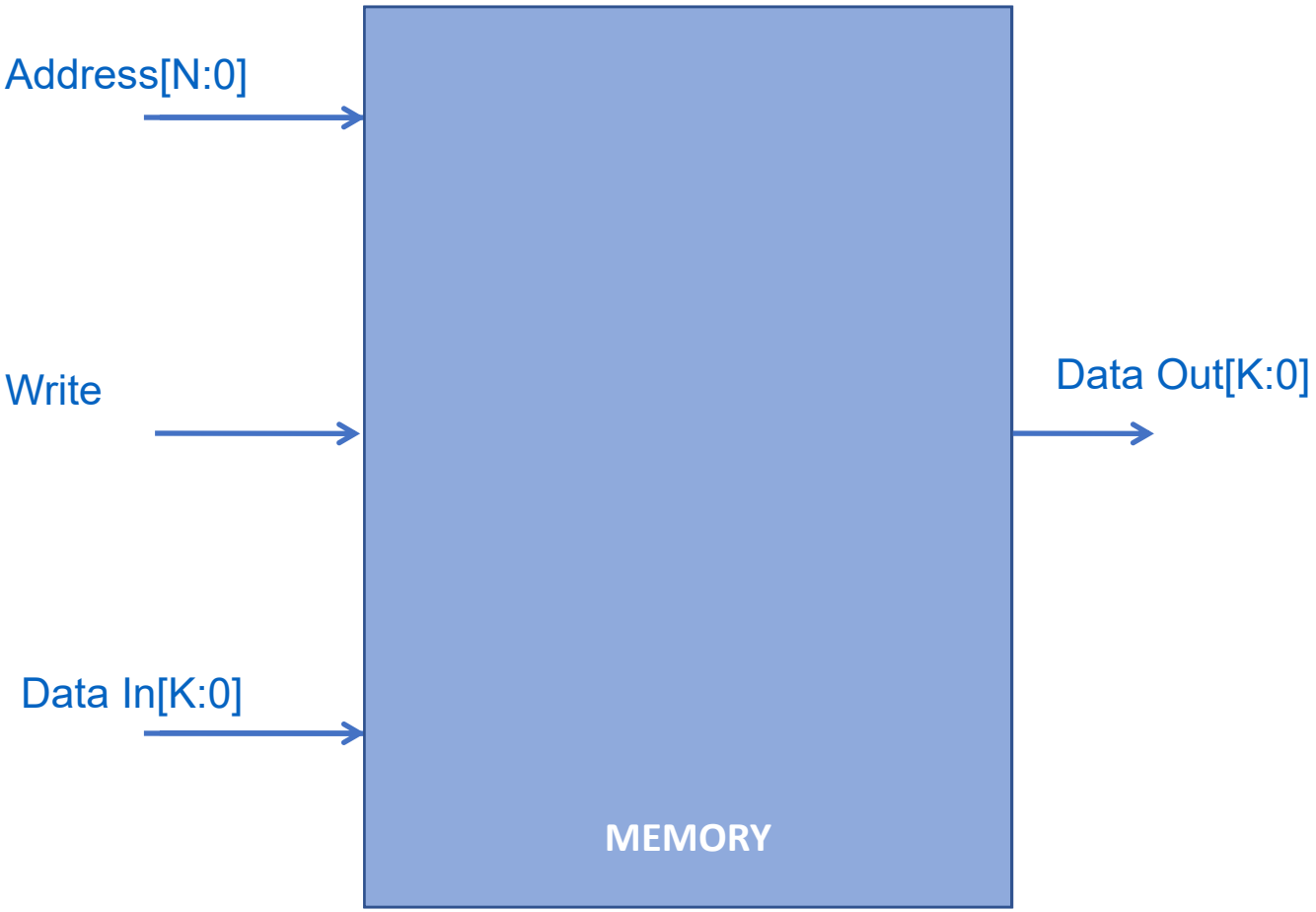




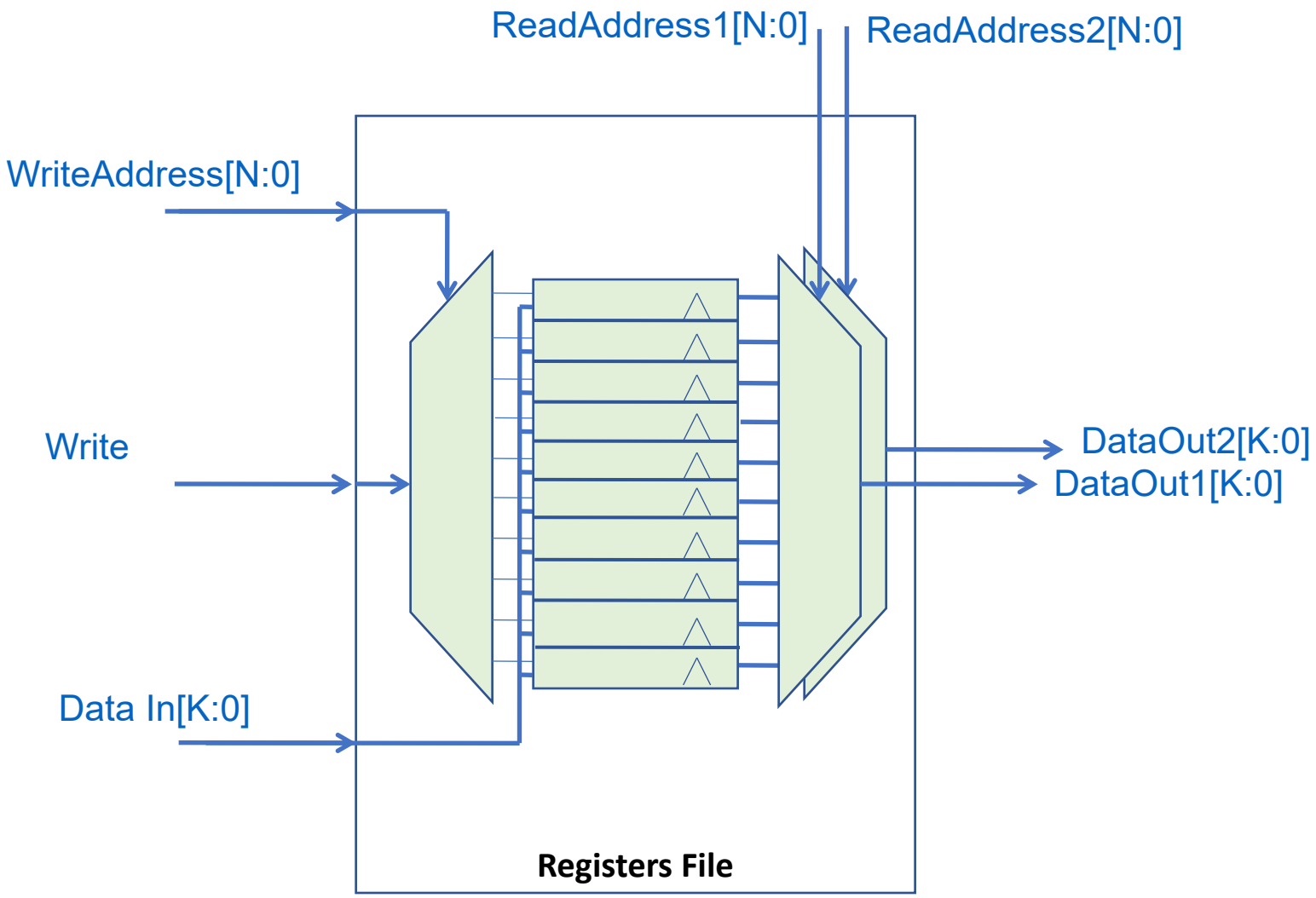

Input[N:0]
Clock
Output[N:0]
Enable


Clock
Input[N:0]
3
4
5
6
Output[N:0]
x
5
Enable

Register Files and Memories

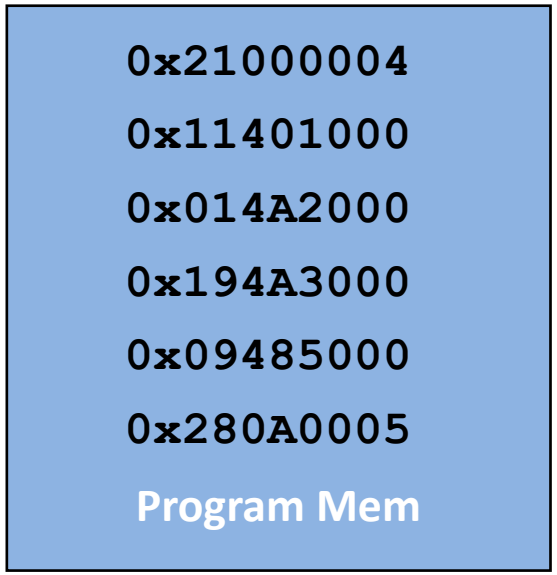


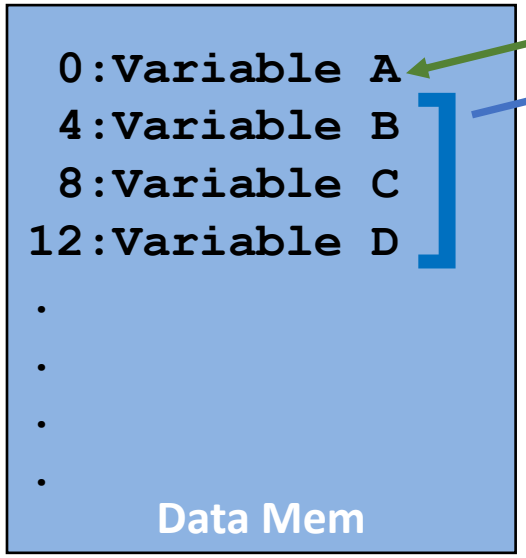
Multiple Read ports

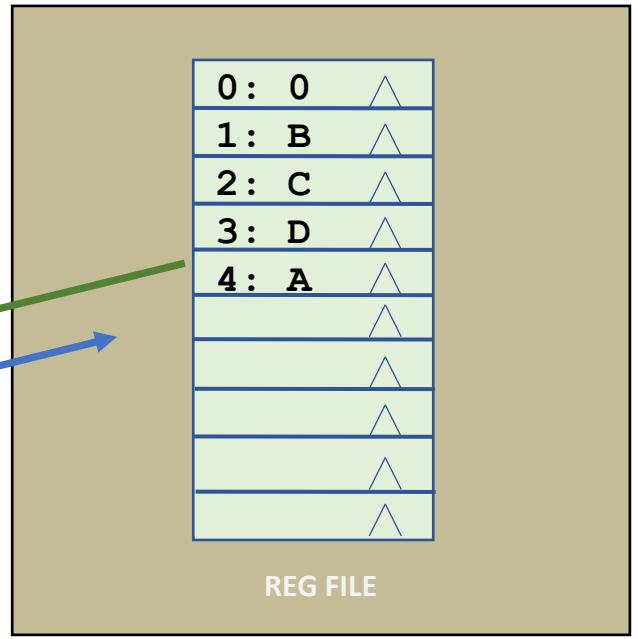


Instruction set (and design)

A = B + (C-D)








```
LW      x1, 4(x0);
LW      x2, 8(x0);
LW      x3, 12(x0);
SUB     x4, x2, x3;
ADD     x4, x1, x4;
SW      x4, 0(x0);
```

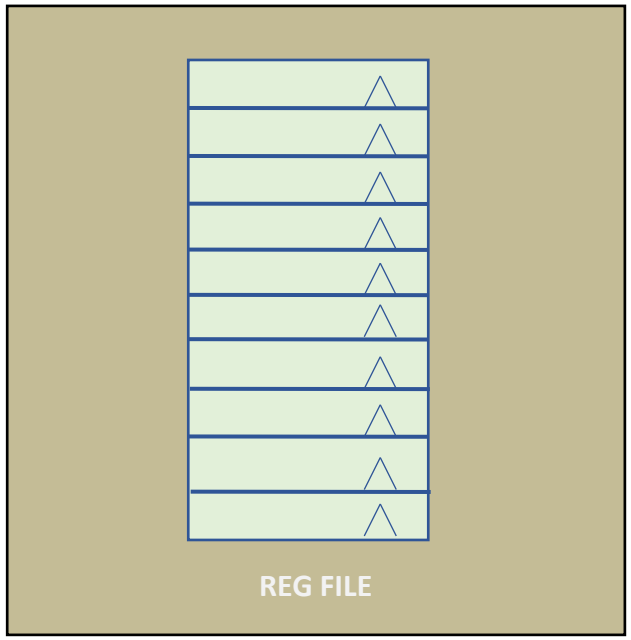
Instruction Set

```
Instruction 1
Instruction 2
Instruction 3
Instruction 4
.
.
.
```

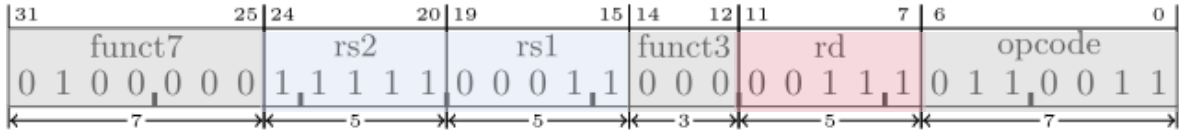
Program Mem

```
 0:Variable A
 4:Variable B
 8:Variable C
12:Variable D
.
.
.
.
```

Data Mem



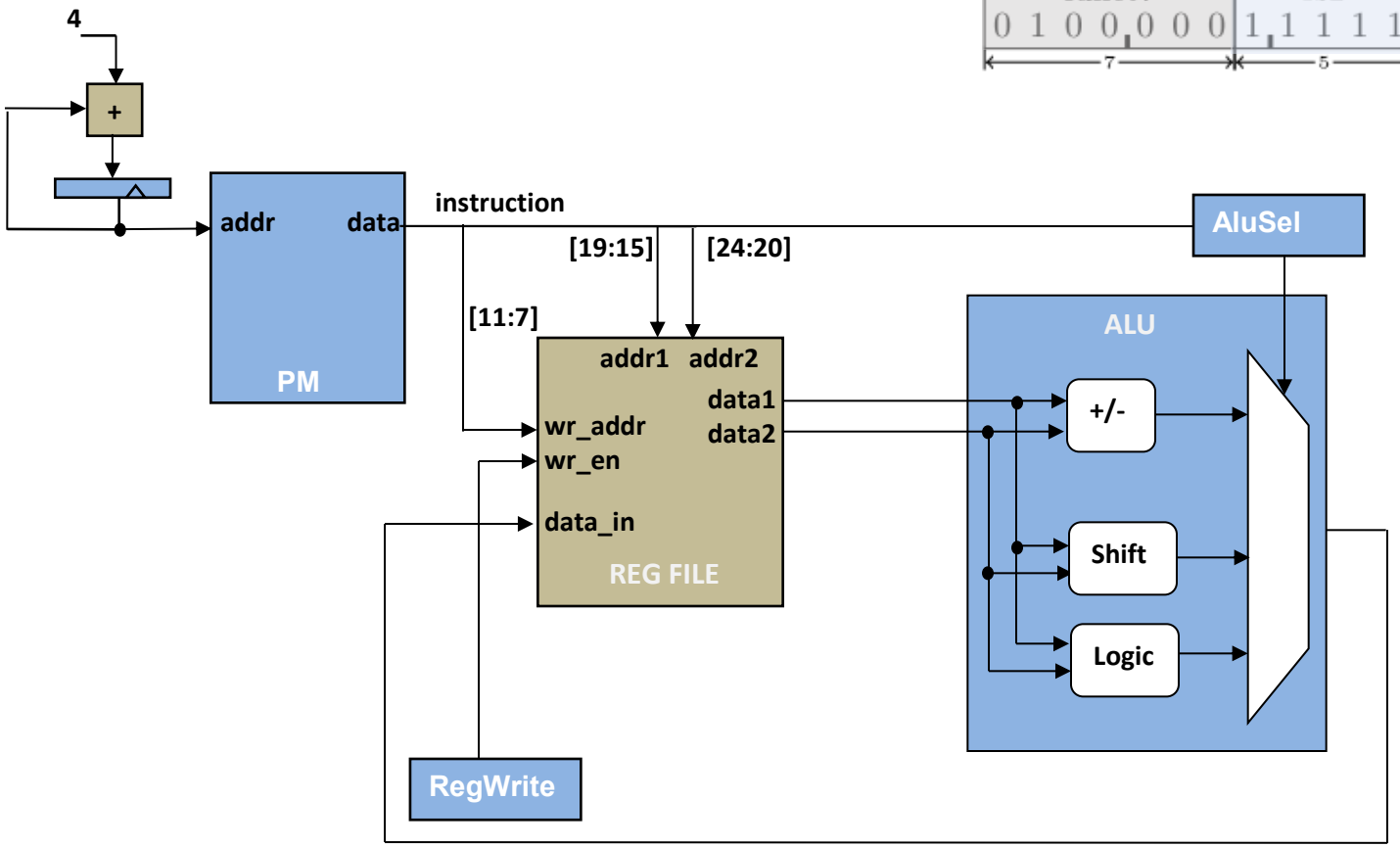
Reg-Reg ALU instructions

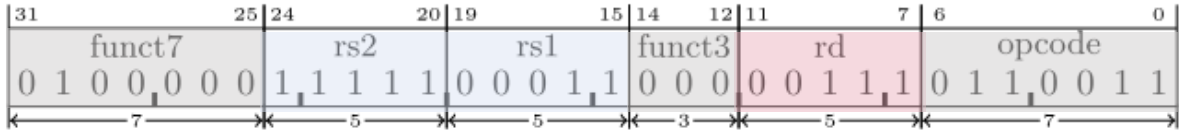


R-type

0 0 0 0 0 0 0	0 0 0	add rd,rs1,rs2
0 1 0 0 0 0 0	0 0 0	sub rd,rs1,rs2
0 0 0 0 0 0 0	0 0 1	sll rd,rs1,rs2
0 0 0 0 0 0 0	0 1 0	slt rd,rs1,rs2
0 0 0 0 0 0 0	0 1 1	sltu rd,rs1,rs2
0 0 0 0 0 0 0	1 0 0	xor rd,rs1,rs2
0 0 0 0 0 0 0	1 0 1	srl rd,rs1,rs2
0 1 0 0 0 0 0	1 0 1	sra rd,rs1,rs2
0 0 0 0 0 0 0	1 1 0	or rd,rs1,rs2
0 0 0 0 0 0 0	1 1 1	and rd,rs1,rs2

Reg-Reg ALU instructions

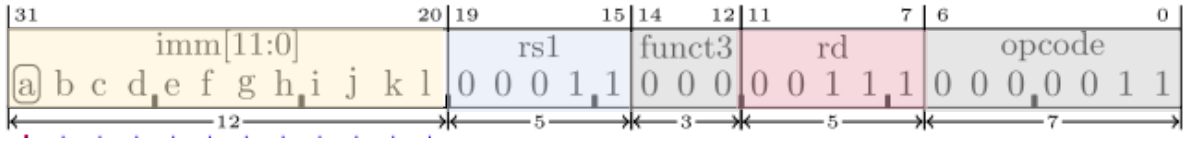




R-type

0 0 0	add	rd,rs1,rs2
0 0 0	sub	rd,rs1,rs2
0 0 1	sll	rd,rs1,rs2
0 1 0	slt	rd,rs1,rs2
0 1 1	sltu	rd,rs1,rs2
1 0 0	xor	rd,rs1,rs2
1 0 1	srl	rd,rs1,rs2
1 0 1	sra	rd,rs1,rs2
1 1 0	or	rd,rs1,rs2
1 1 1	and	rd,rs1,rs2

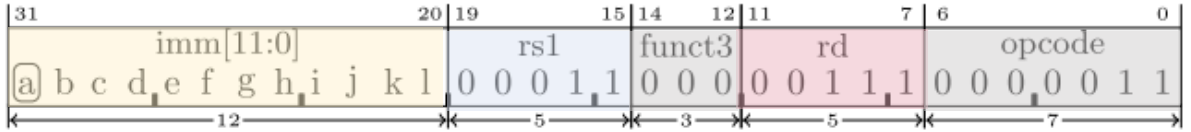
Load instructions



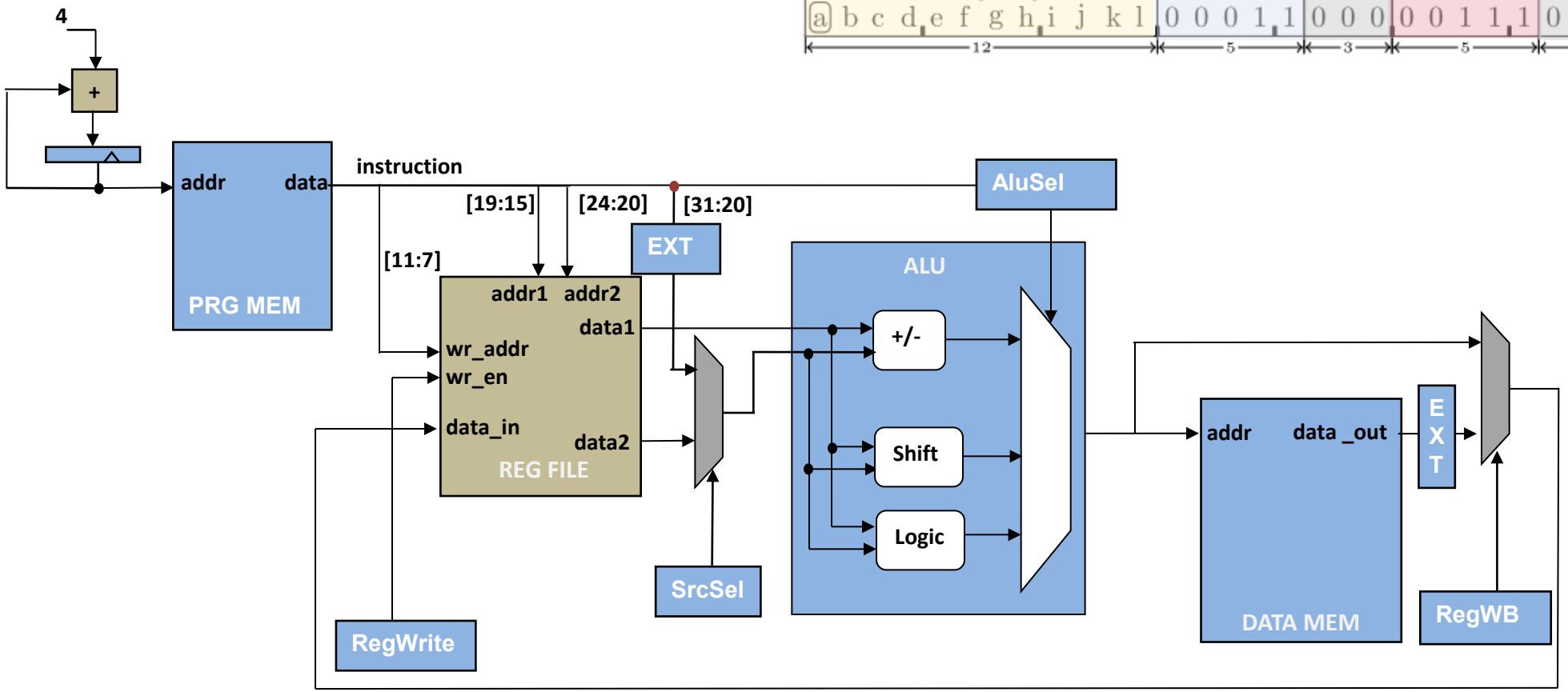
I-type

0 1 0	`lw`	`rd,imm(rs1)`
1 0 0	`lbu`	`rd,imm(rs1)`
1 0 1	`lhu`	`rd,imm(rs1)`

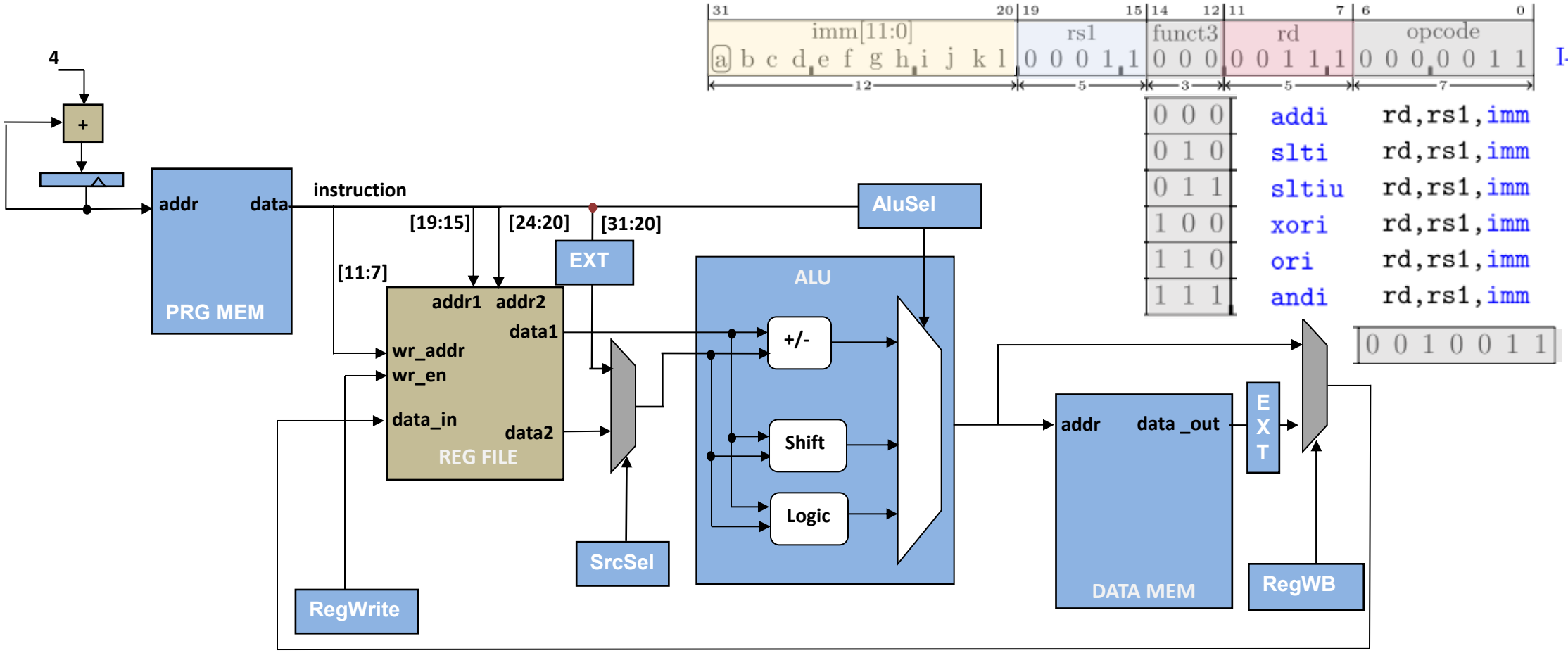
Load instructions



I-type



Alu-Imm instructions



0 0 0	addi	rd,rs1,imm
0 1 0	slti	rd,rs1,imm
0 1 1	sltiu	rd,rs1,imm
1 0 0	xori	rd,rs1,imm
1 1 0	ori	rd,rs1,imm
1 1 1	andi	rd,rs1,imm

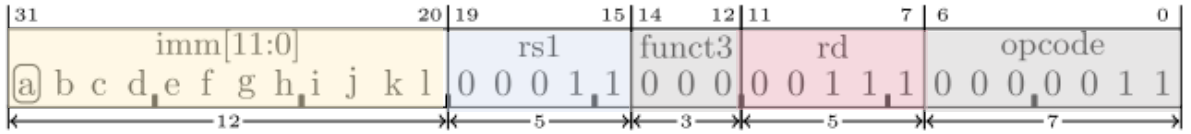
0 0 1 0 0 1 1

```
for( int i=0; x <= 10; i++)
{
     Some piece of code;
}
```

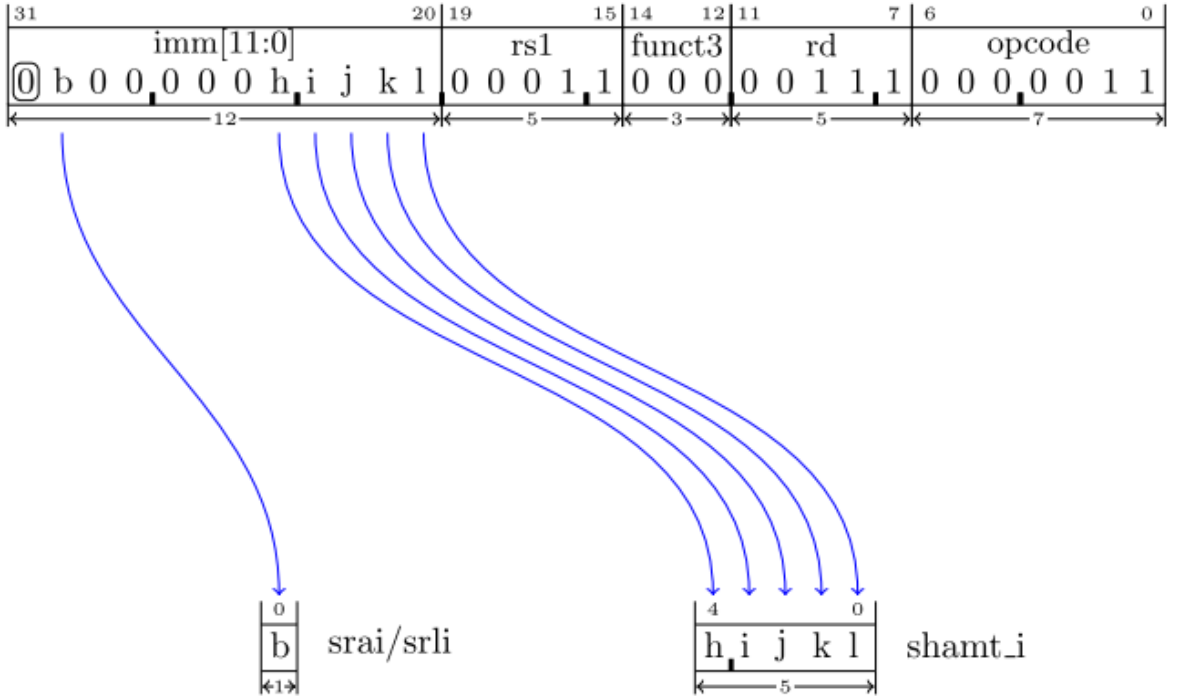
```
i--;
i+=10; // i = i + 10
int x = 10;
```

```
addi x1, x1, 10;
addi x1, x0, 10;
```

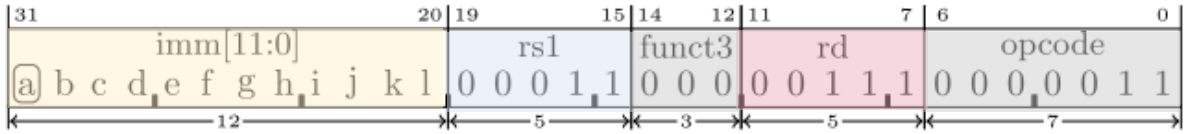
Alu-Imm instructions




```
slli    rd,rs1,shamt
srli    rd,rs1,shamt
srai    rd,rs1,shamt
```

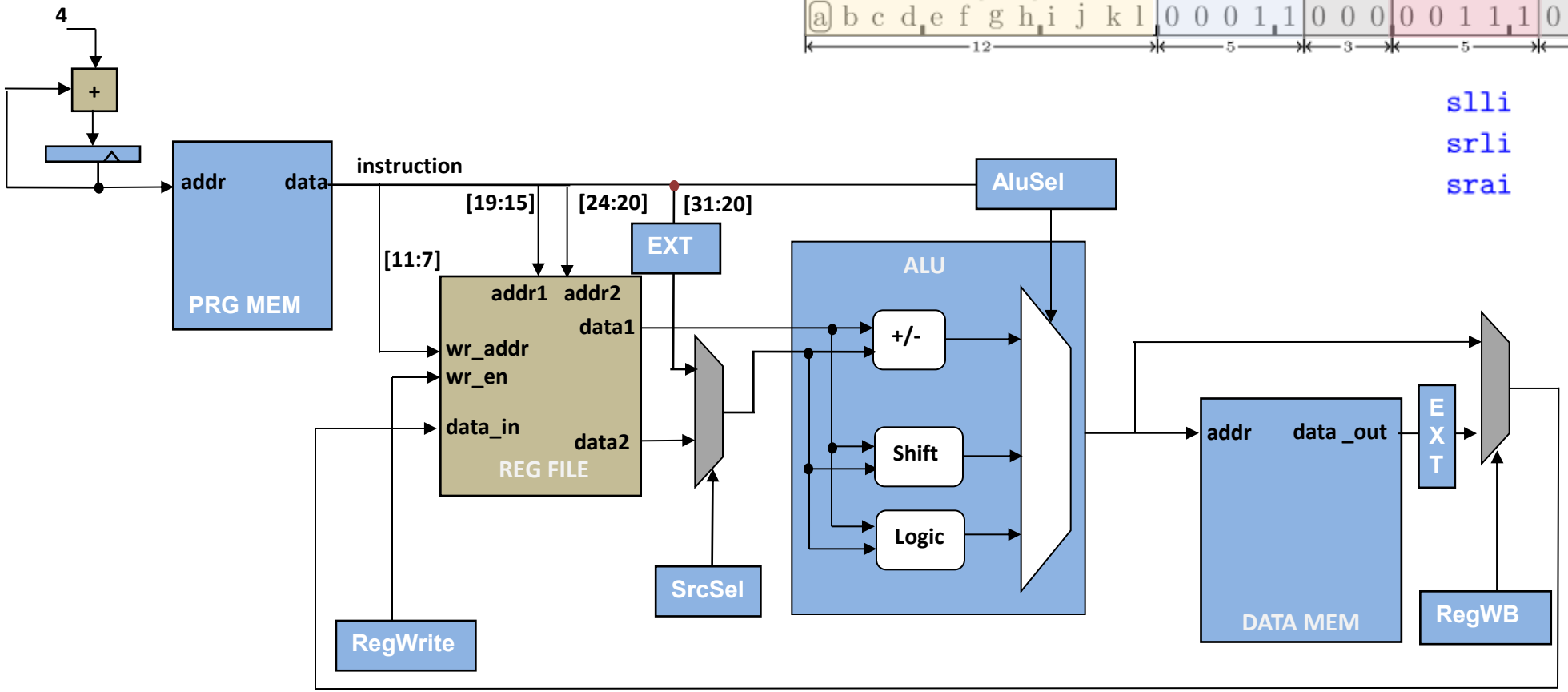


Alu-Imm instructions



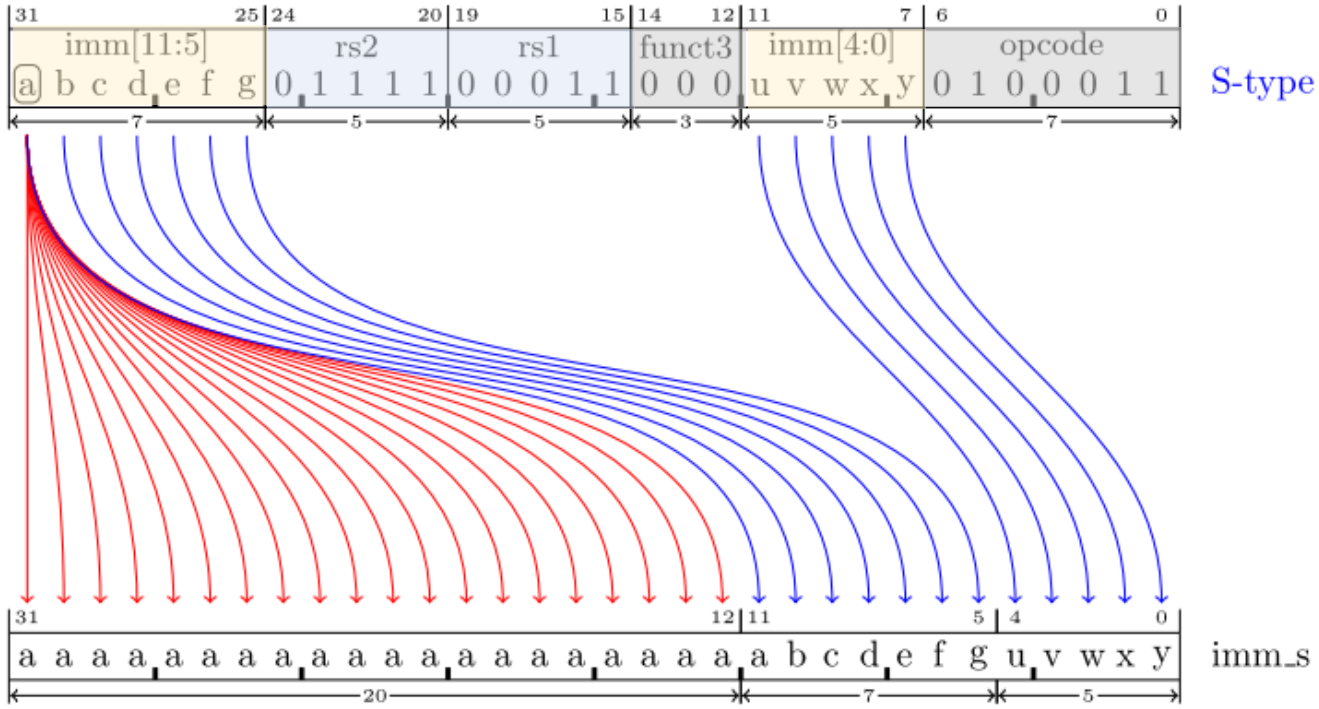
I-type

```
slli    rd,rs1,shamt
srli    rd,rs1,shamt
srai    rd,rs1,shamt
```



Store instructions

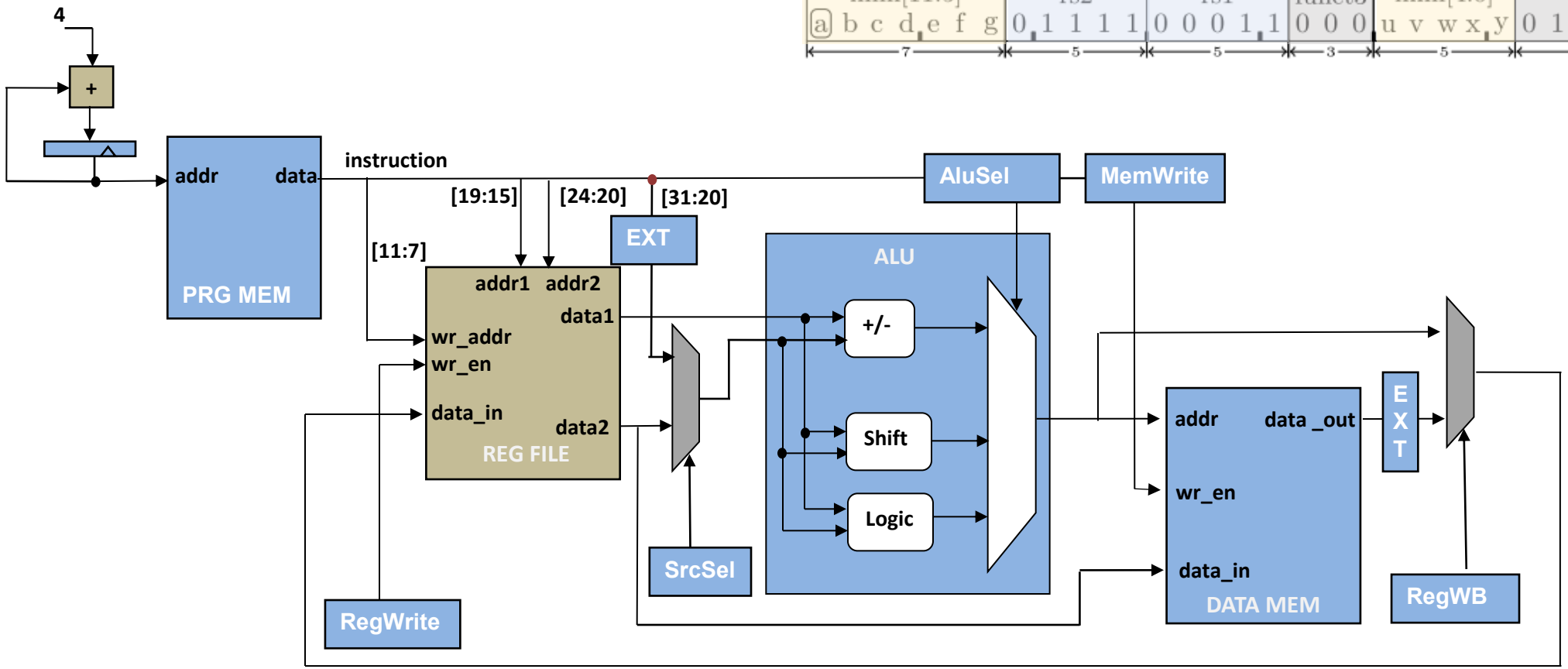
0 0 0	sb	rs2,imm(rs1)
0 0 1	sh	rs2,imm(rs1)
0 1 0	sw	rs2,imm(rs1)

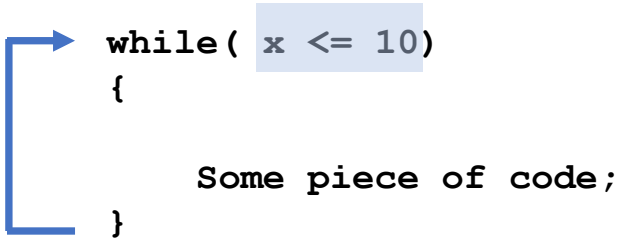


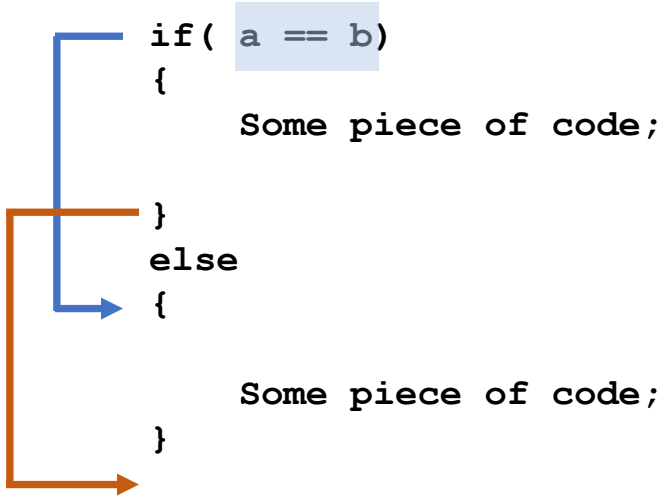
Store instructions



S-type

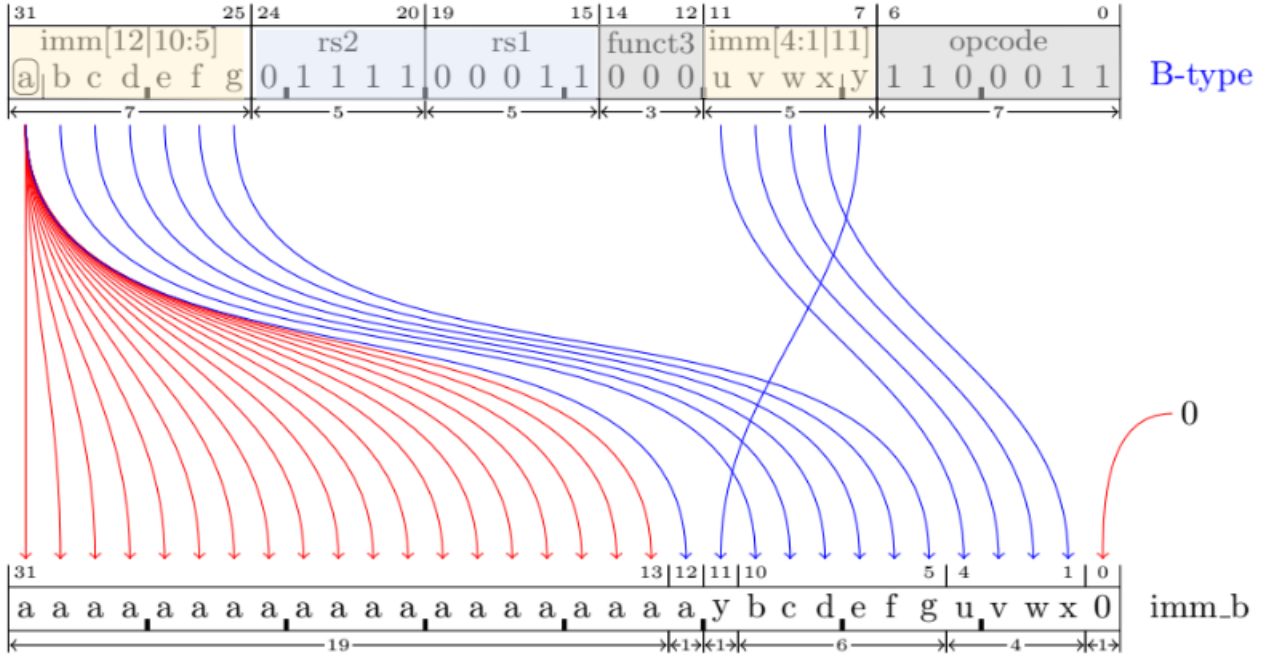



while(x <= 10)
{
Some piece of code;
}

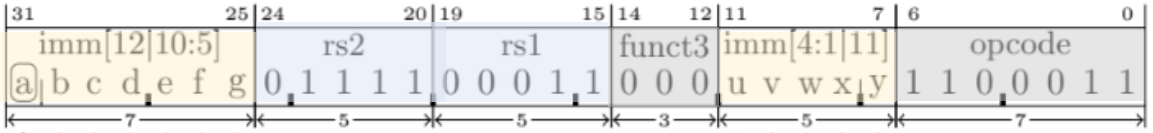

if(a == b)
{
Some piece of code;
}
else
{
Some piece of code;
}

Branch instructions

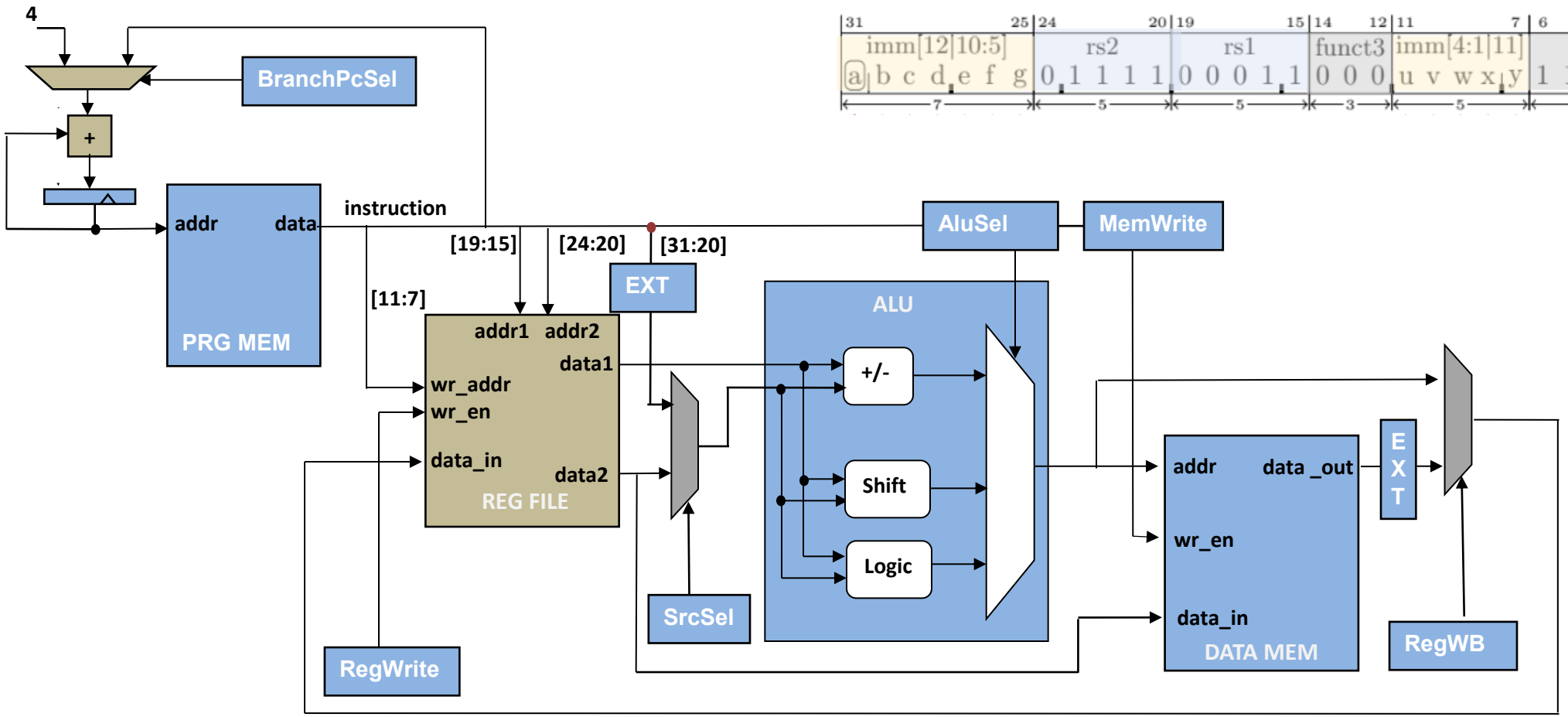
0 0 0	beq	rs1,rs2,pcrel_13
0 0 1	bne	rs1,rs2,pcrel_13
1 0 0	blt	rs1,rs2,pcrel_13
1 0 1	bge	rs1,rs2,pcrel_13
1 1 0	bltu	rs1,rs2,pcrel_13
1 1 1	bgeu	rs1,rs2,pcrel_13



Branch instructions



B-type



Jump instruction

```
int main()
{
    Some piece of code;
    x = factorial(10);
    Some piece of code;
    y = factorial(5);
    Some piece of code;
}
int factorial(int n)
{
    Some piece of code;
    return x;
}
```

Store PC while jumping

Jump back to stored PC+4 using Jalr (coming up)

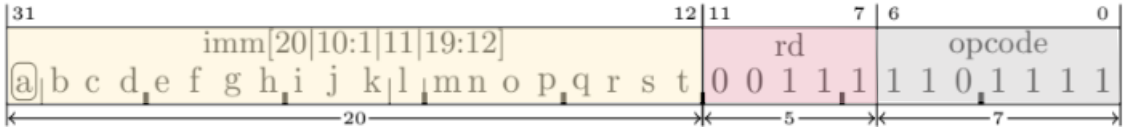
```
jal     rd,pcrel_21
```

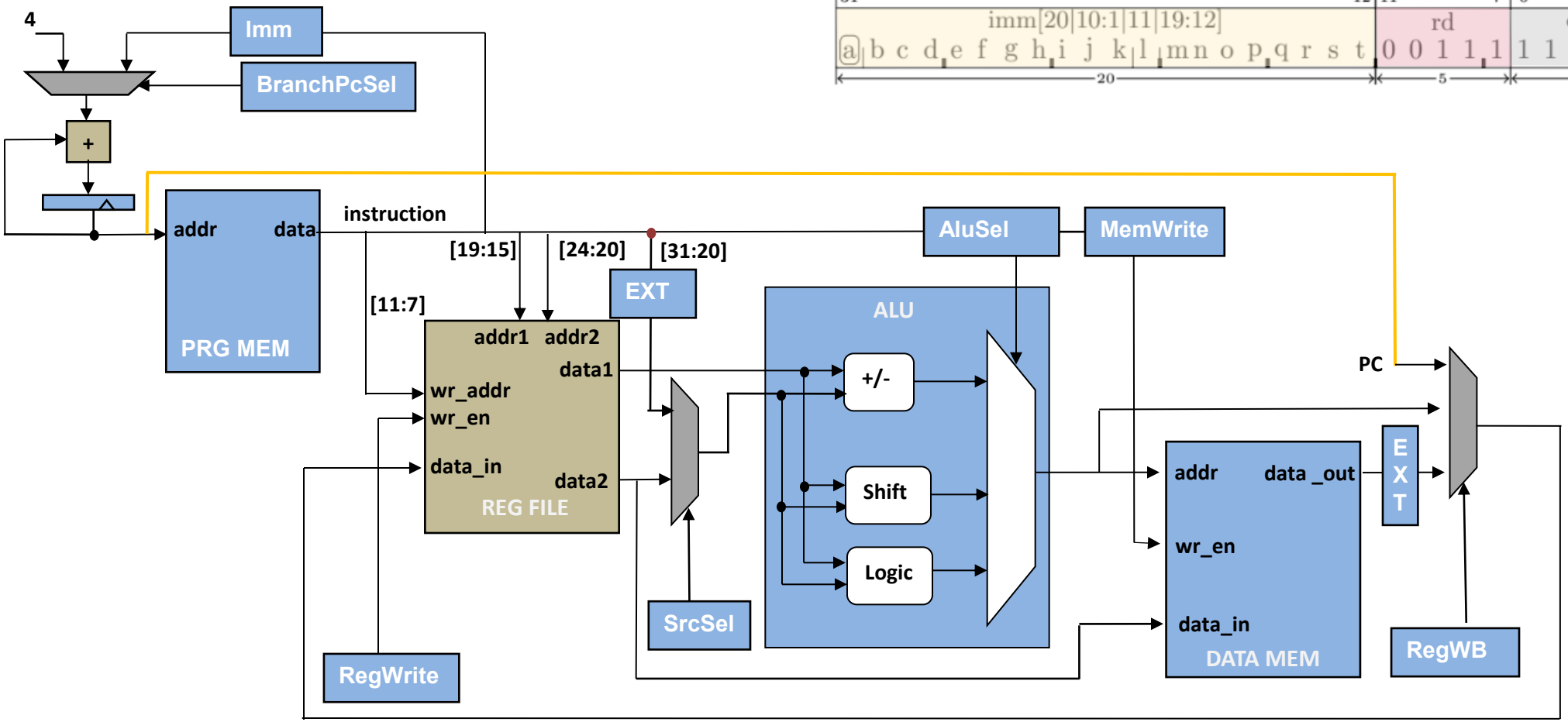
```
if( a == b)
{
    Some piece of code;

}
else
{

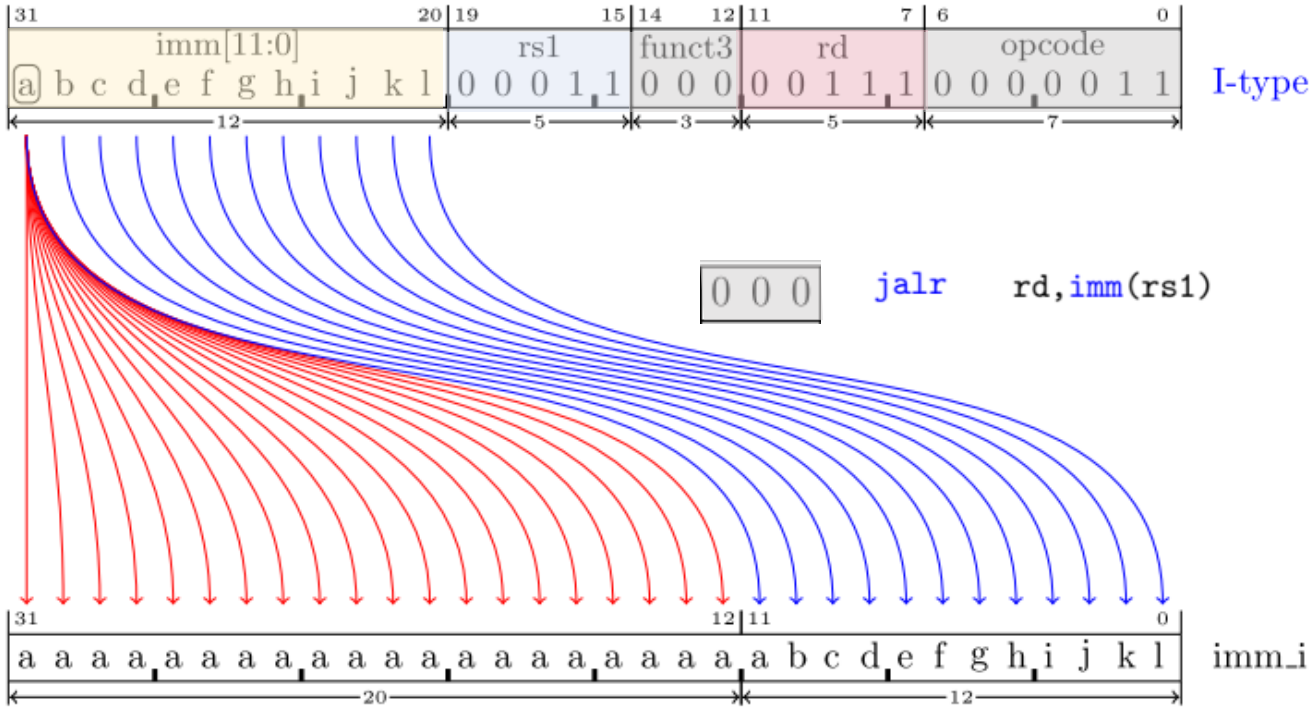
    Some piece of code;
}
```

Jump instruction

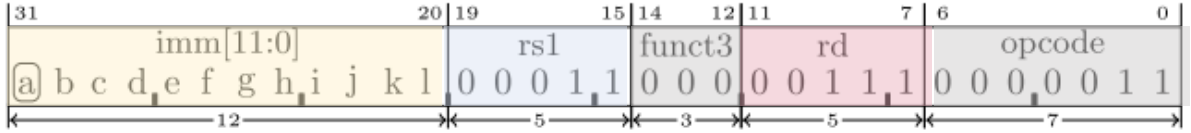




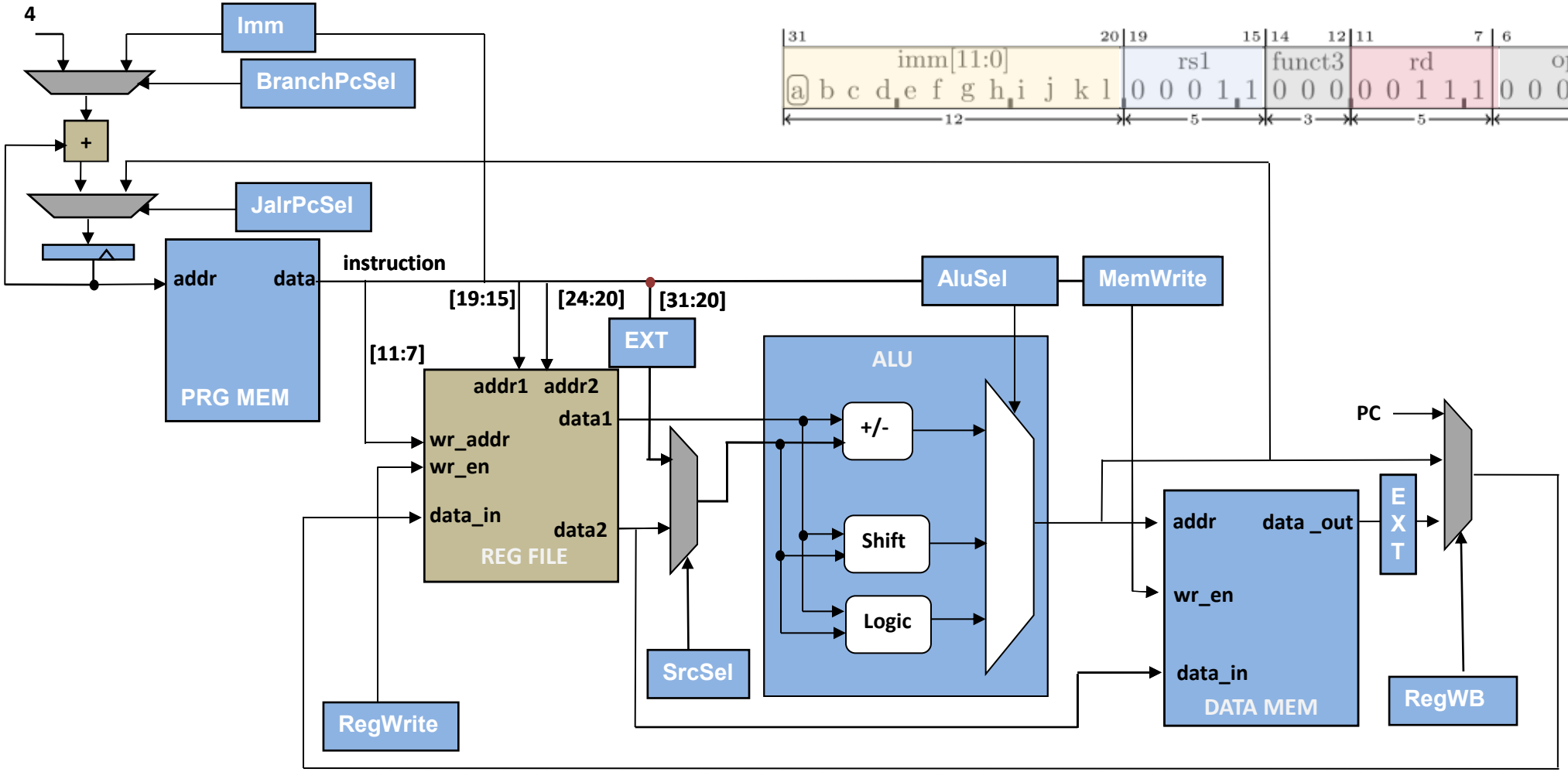
Jalr instruction



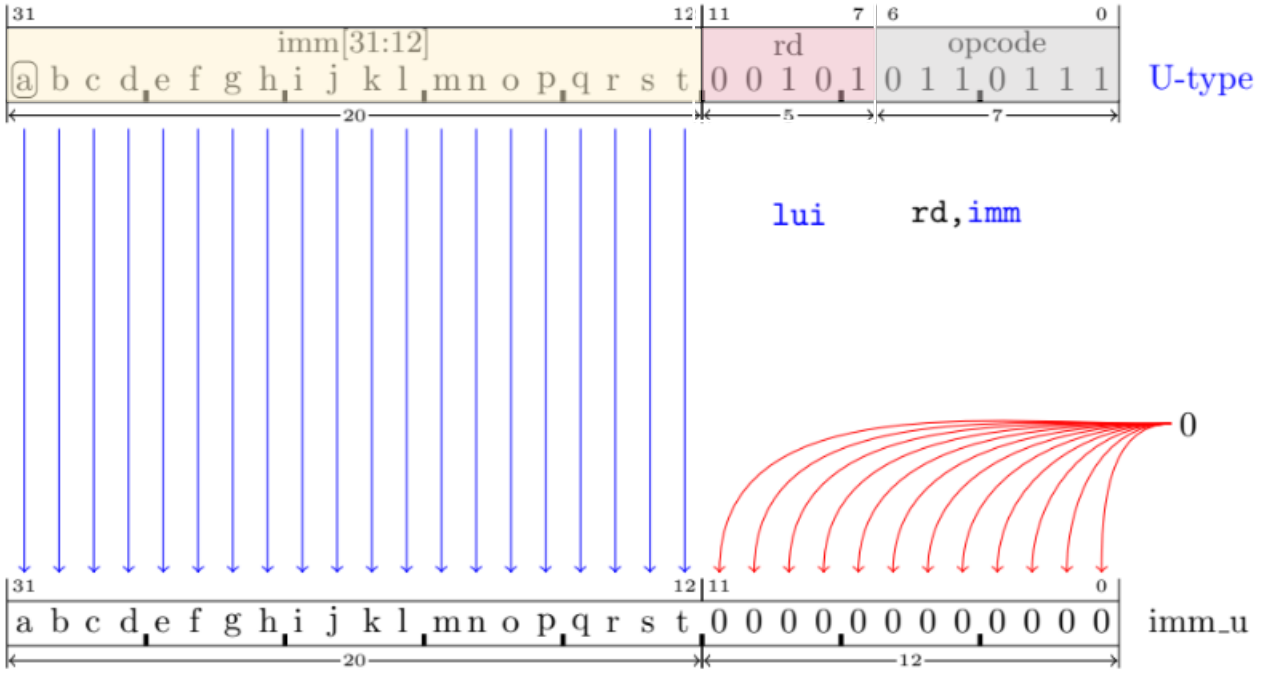
Jalr instruction



I-type



Lui instruction

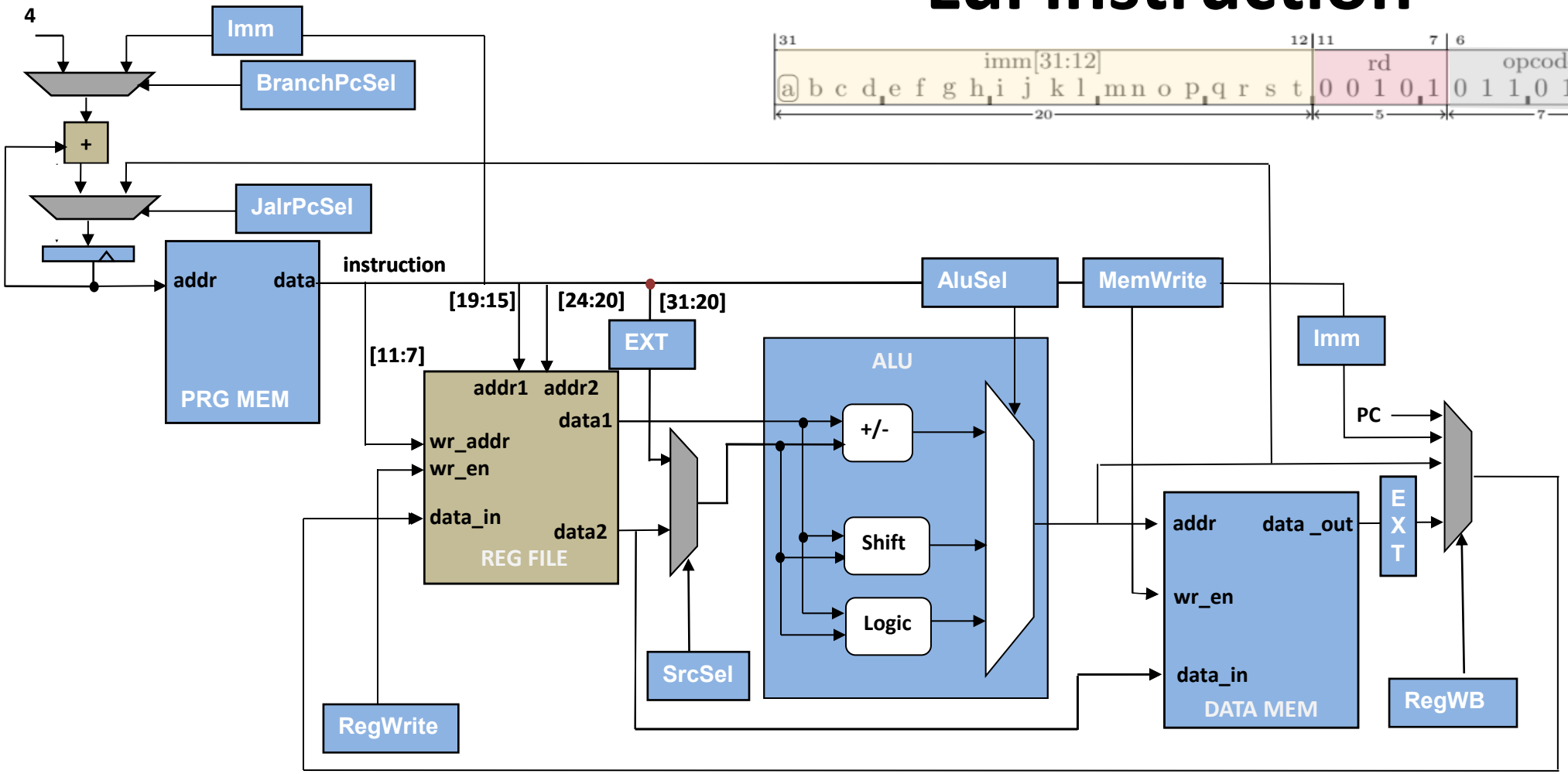



```
                                addi x1, x0, 0xBAD;

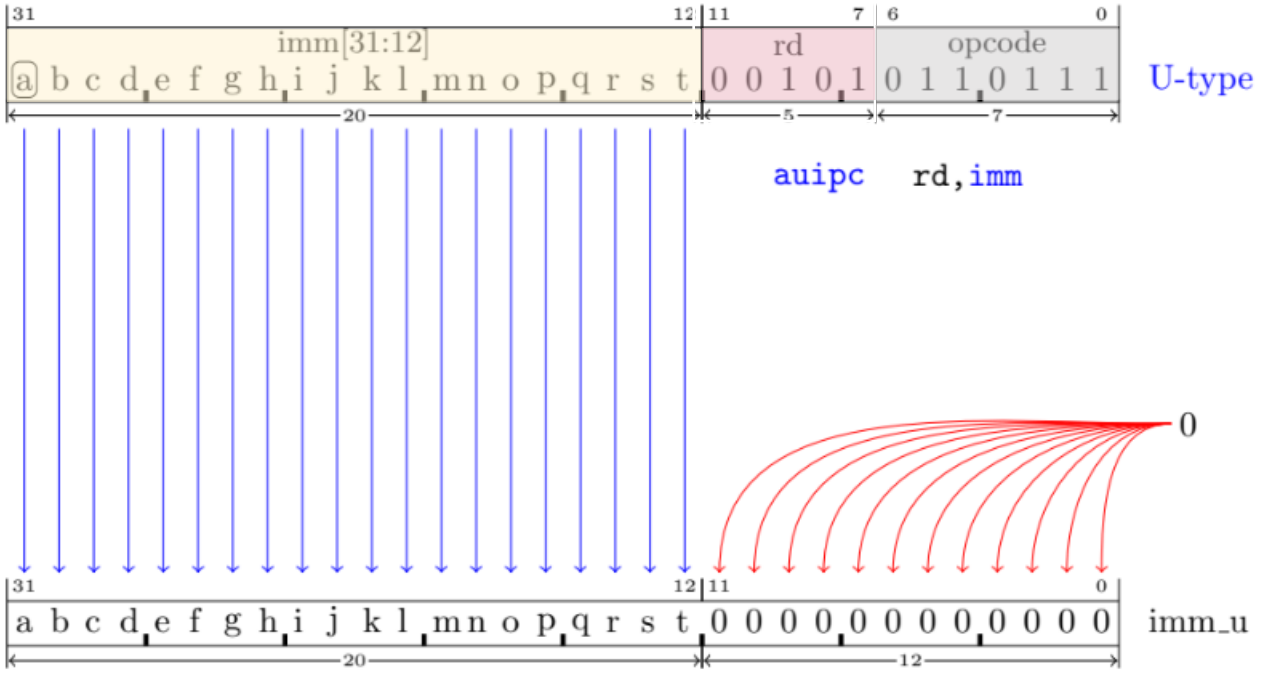
int x = 0xD00D2BAD;             lui  x1, 0xD00D2;          li x1, 0xD00D2BAD;
int x = 0xBAD;                  addi x1, x1, 0xBAD;
```

Lui instruction

U-type



Auipc instruction

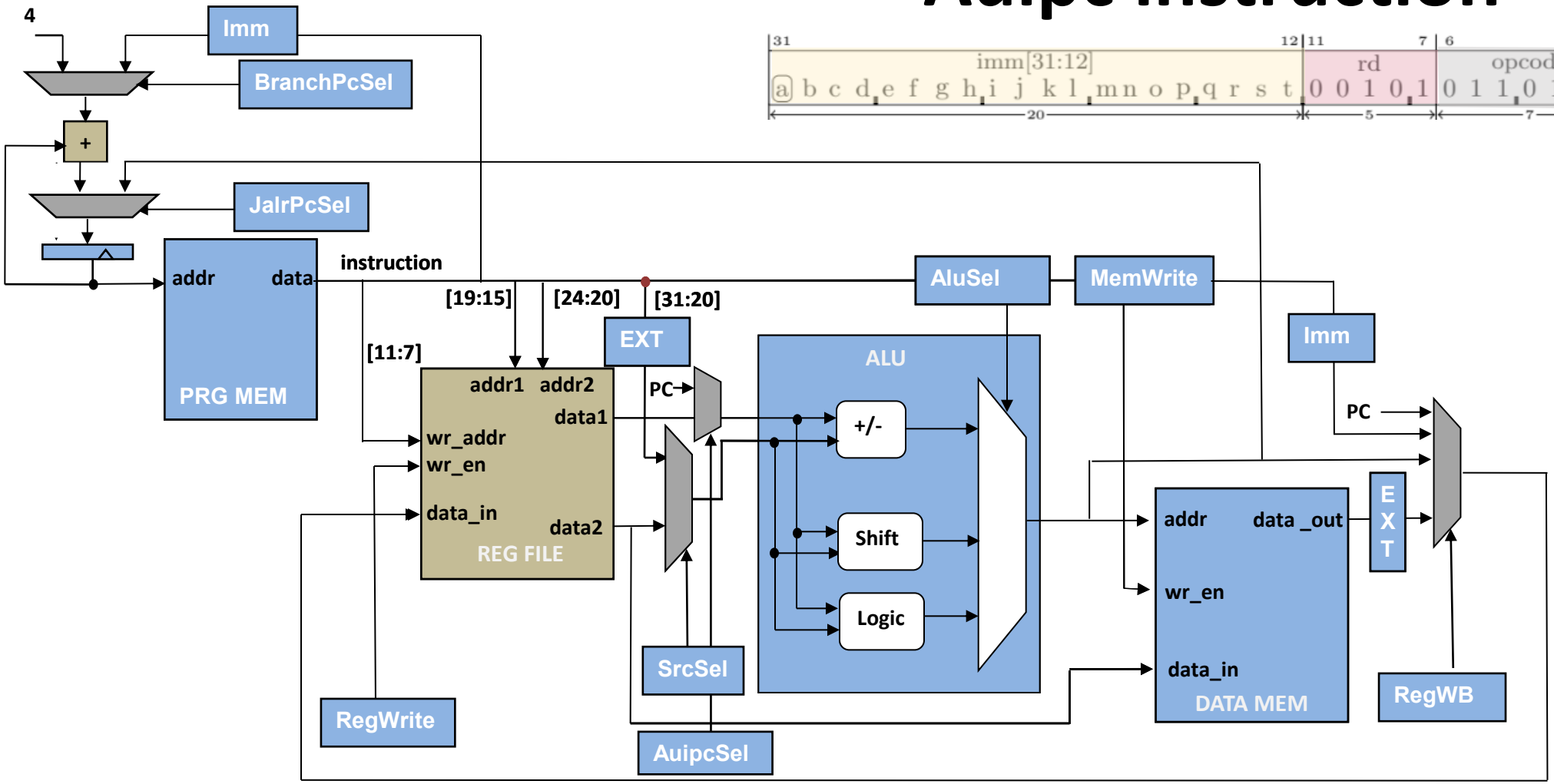



```
auipc  x1, 0x12345;
jalr   x1, x1, 0x678;
```

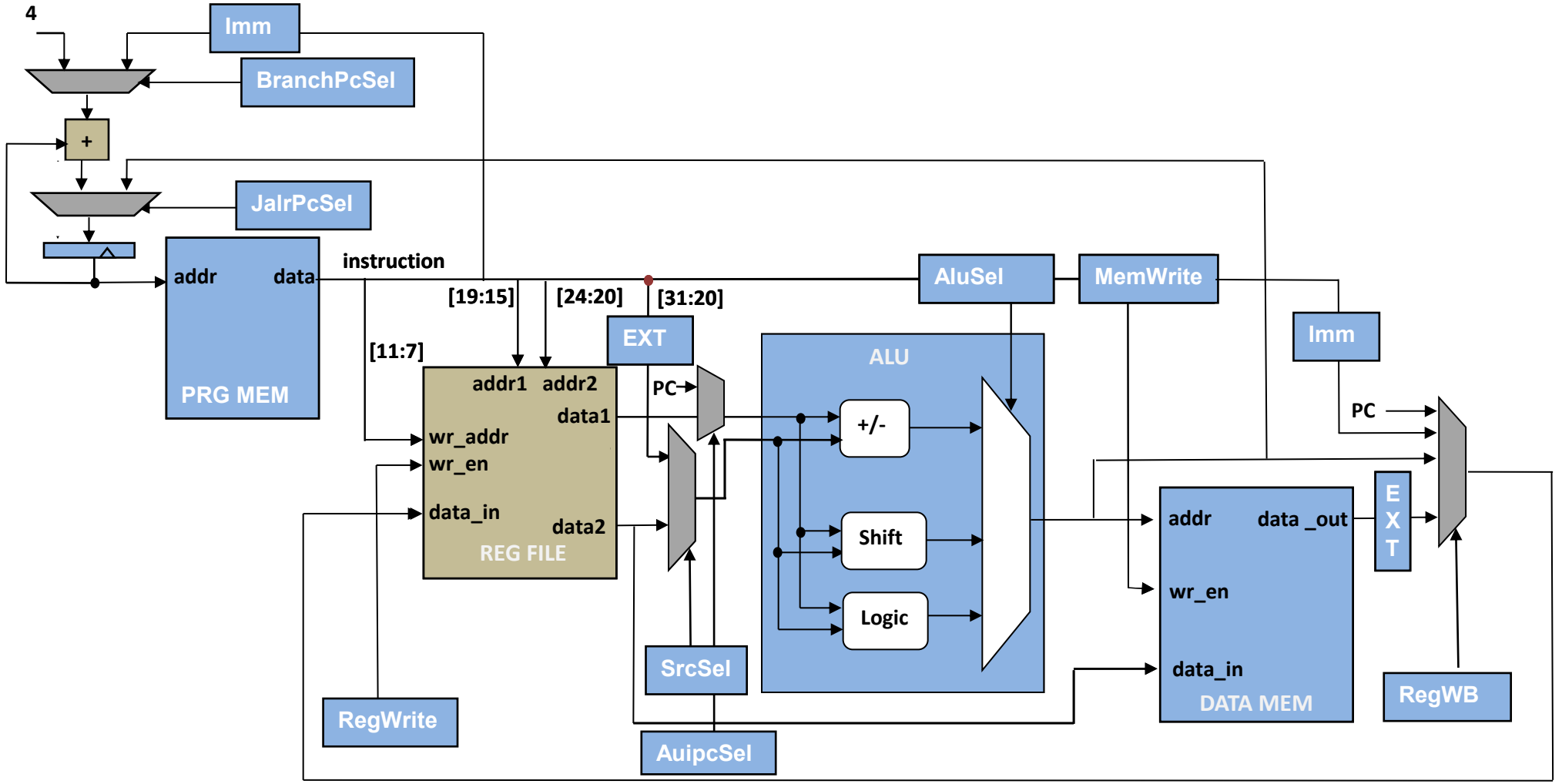
```
call label; // label address PC+0x12345678
```

Auipc instruction

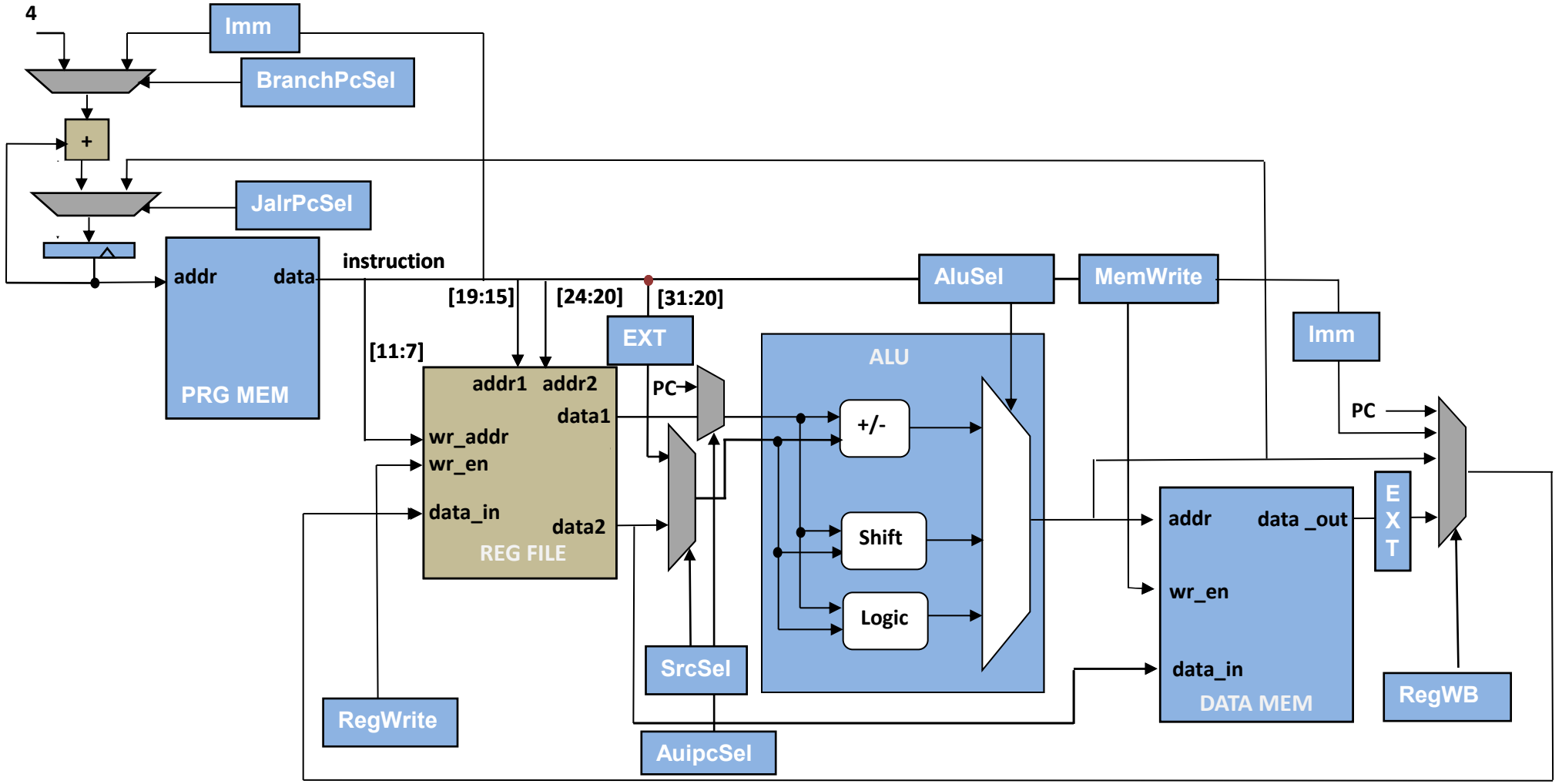
U-type



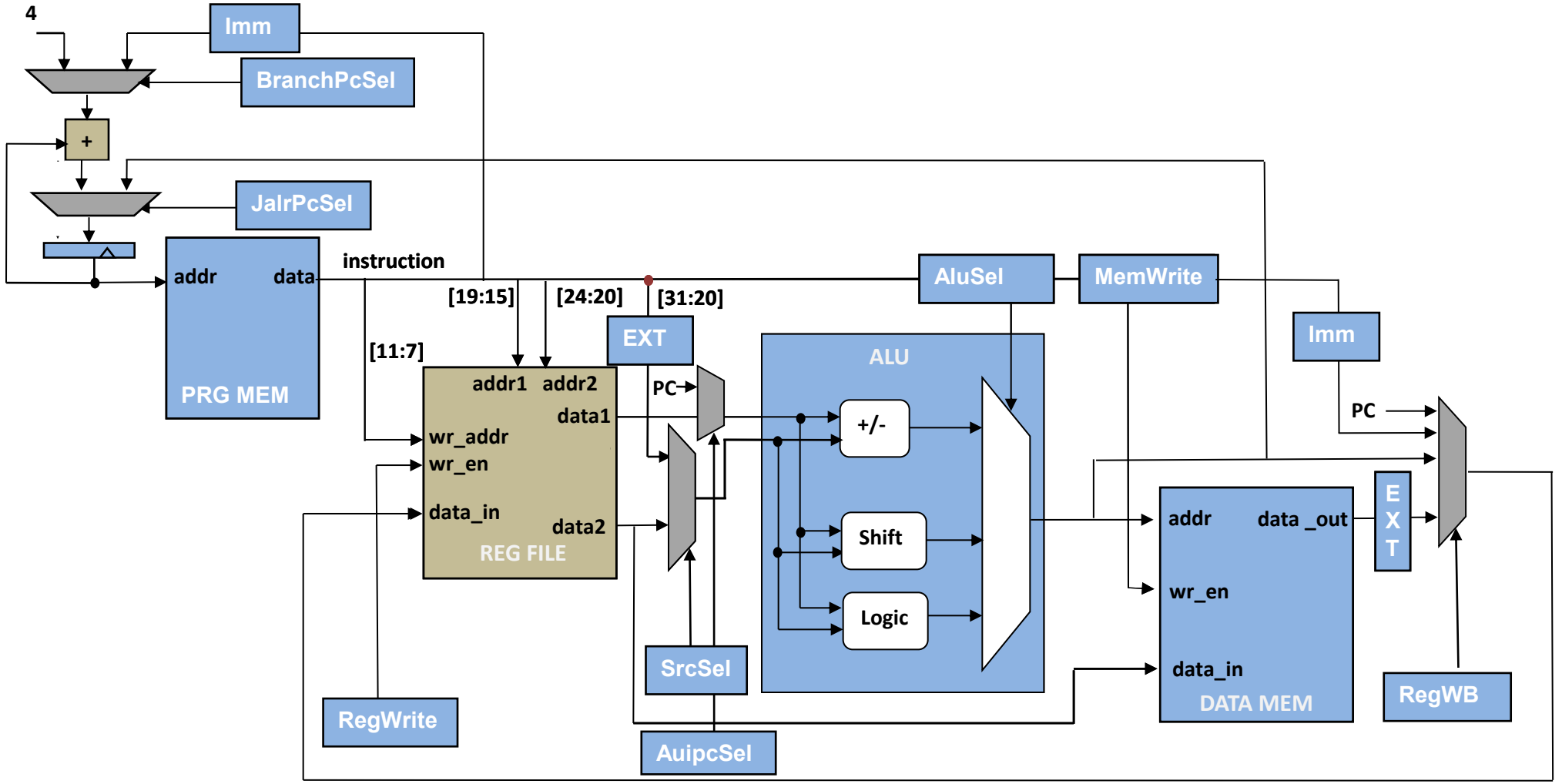
Control Logic



SrcSel	
Alu	Reg
Alui	Imm
Lw	Imm
Sw	Imm
Branch T	Reg
Branch F	Reg
Jal	x
Jalr	Imm
Auipc	Imm
Lui	x

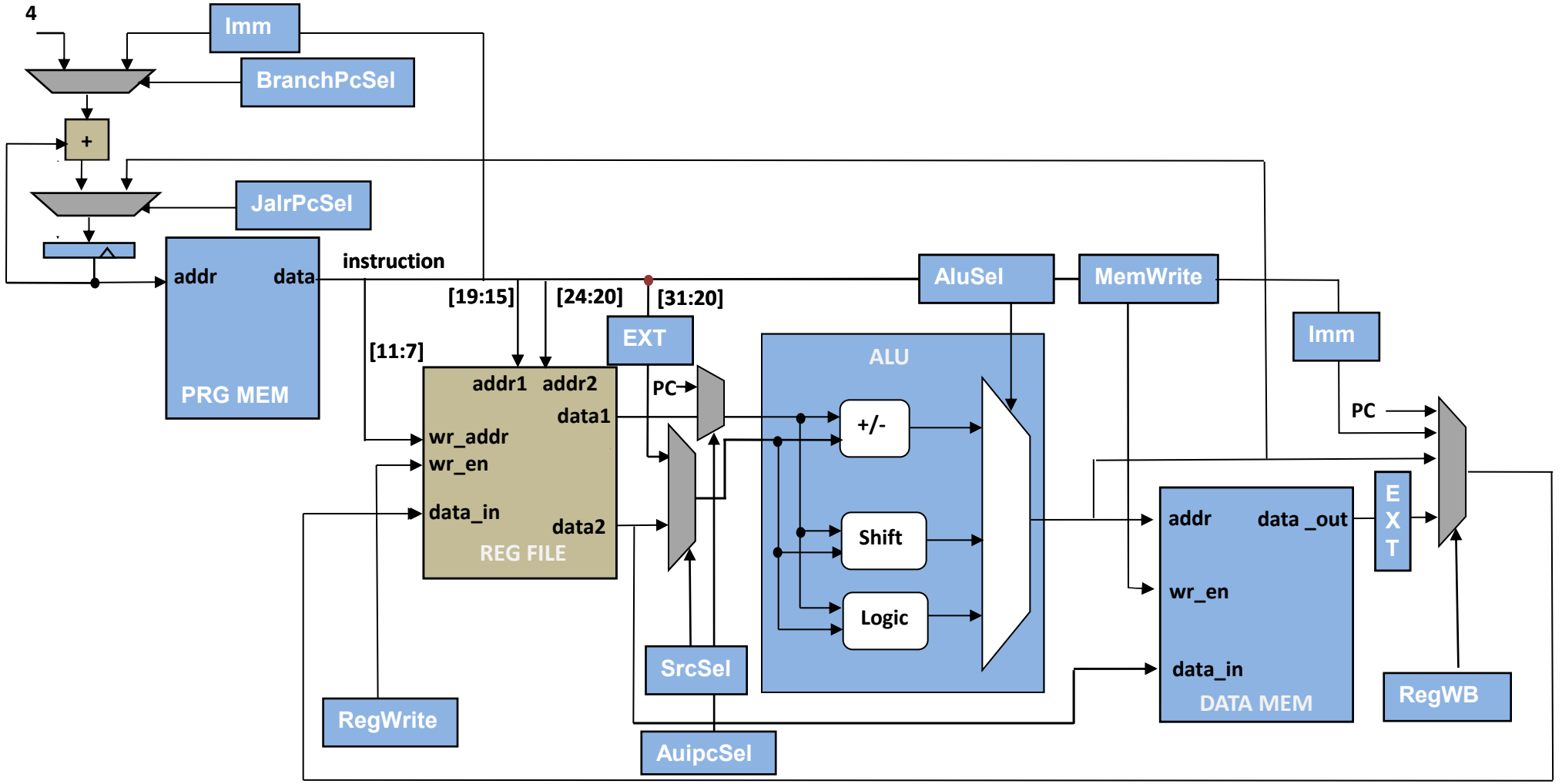


MemWrite	
Alu	No
Alui	No
Lw	No
Sw	Yes
Branch T	No
Branch F	No
Jal	No
Jalr	No
Auipc	No
Lui	No

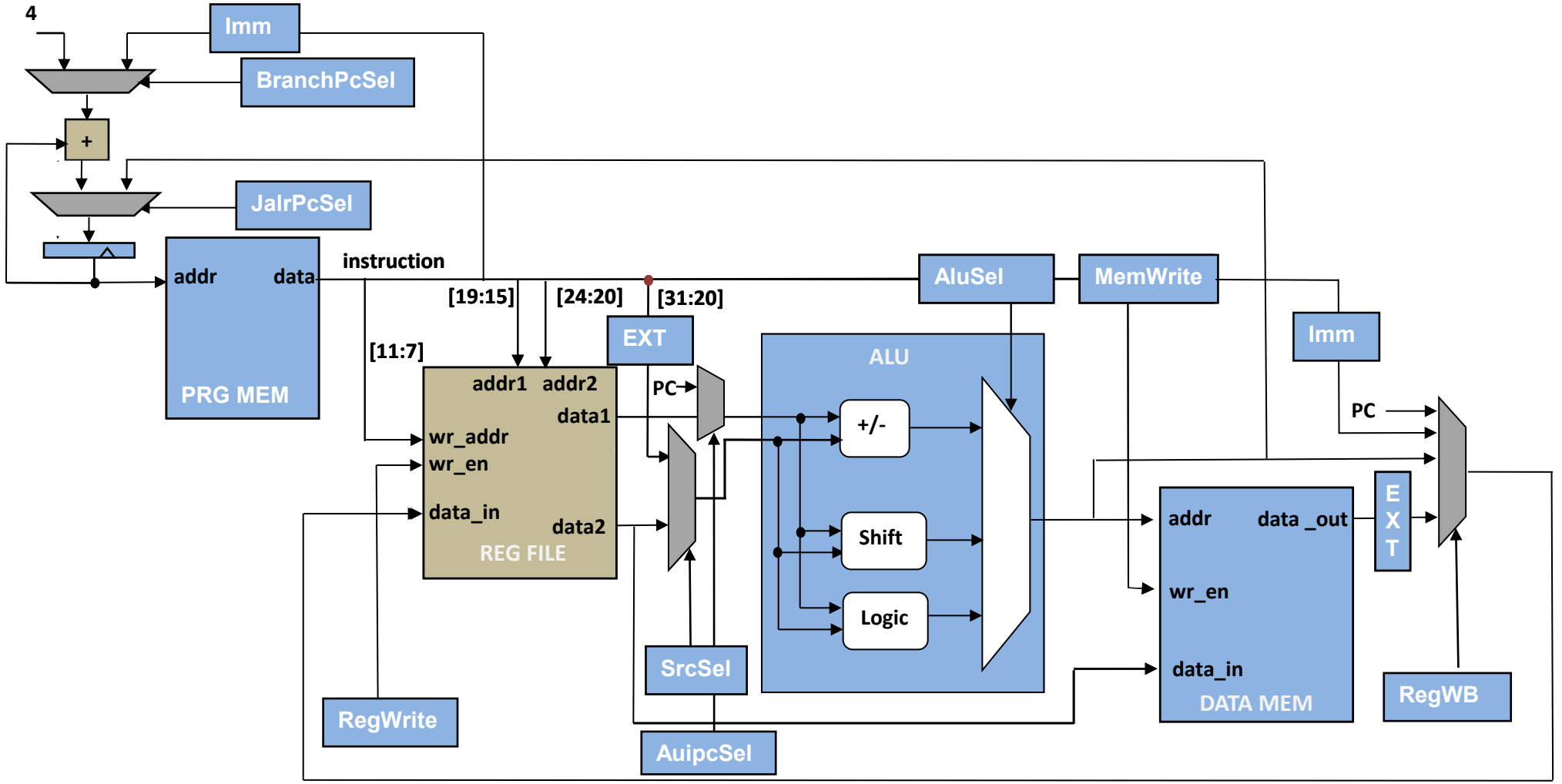


RegWrite

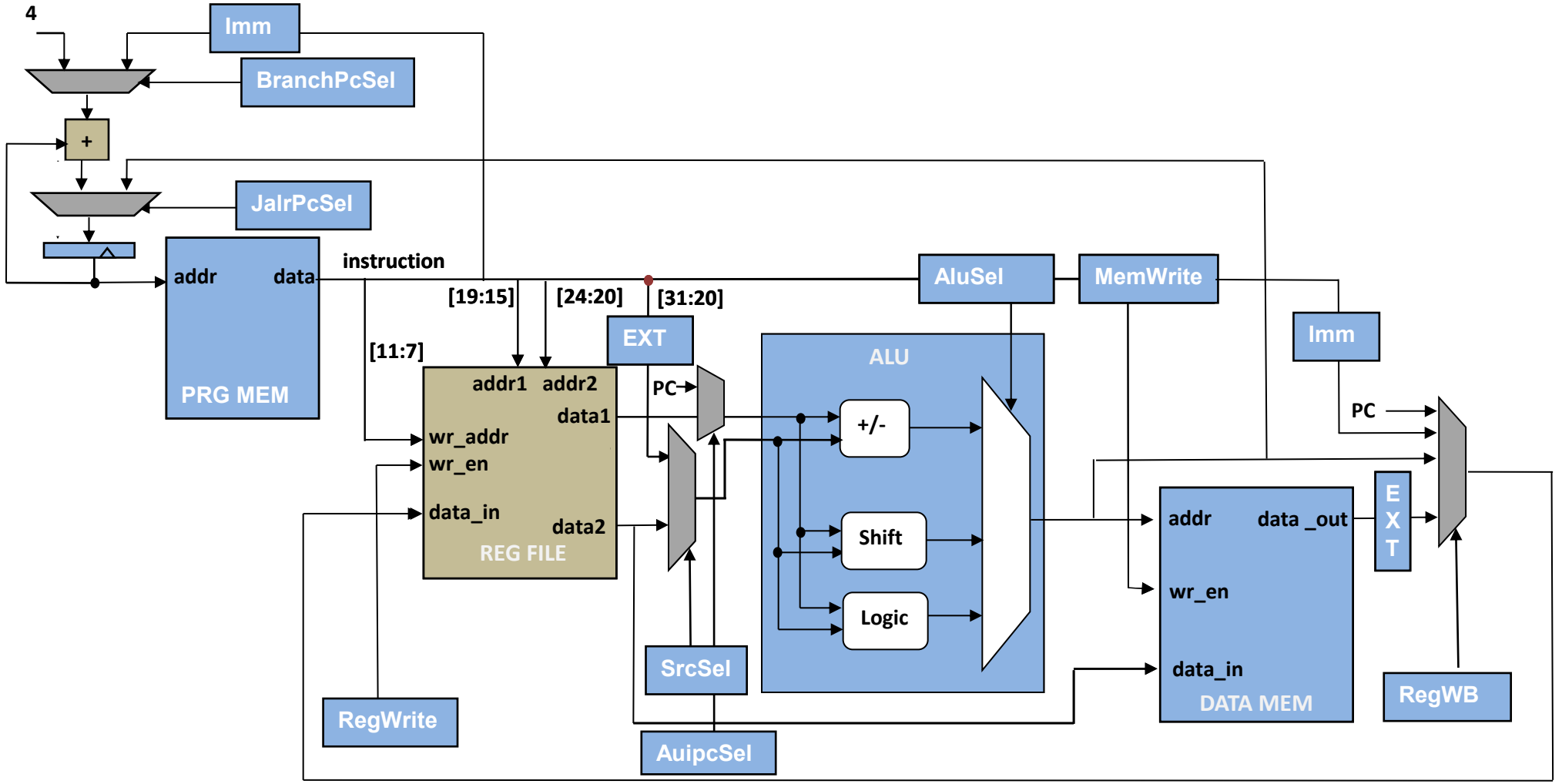
Alu	Yes
Alui	Yes
Lw	Yes
Sw	No
Branch T	No
Branch F	No
Jal	Yes
Jalr	Yes
Auipc	Yes
Lui	Yes



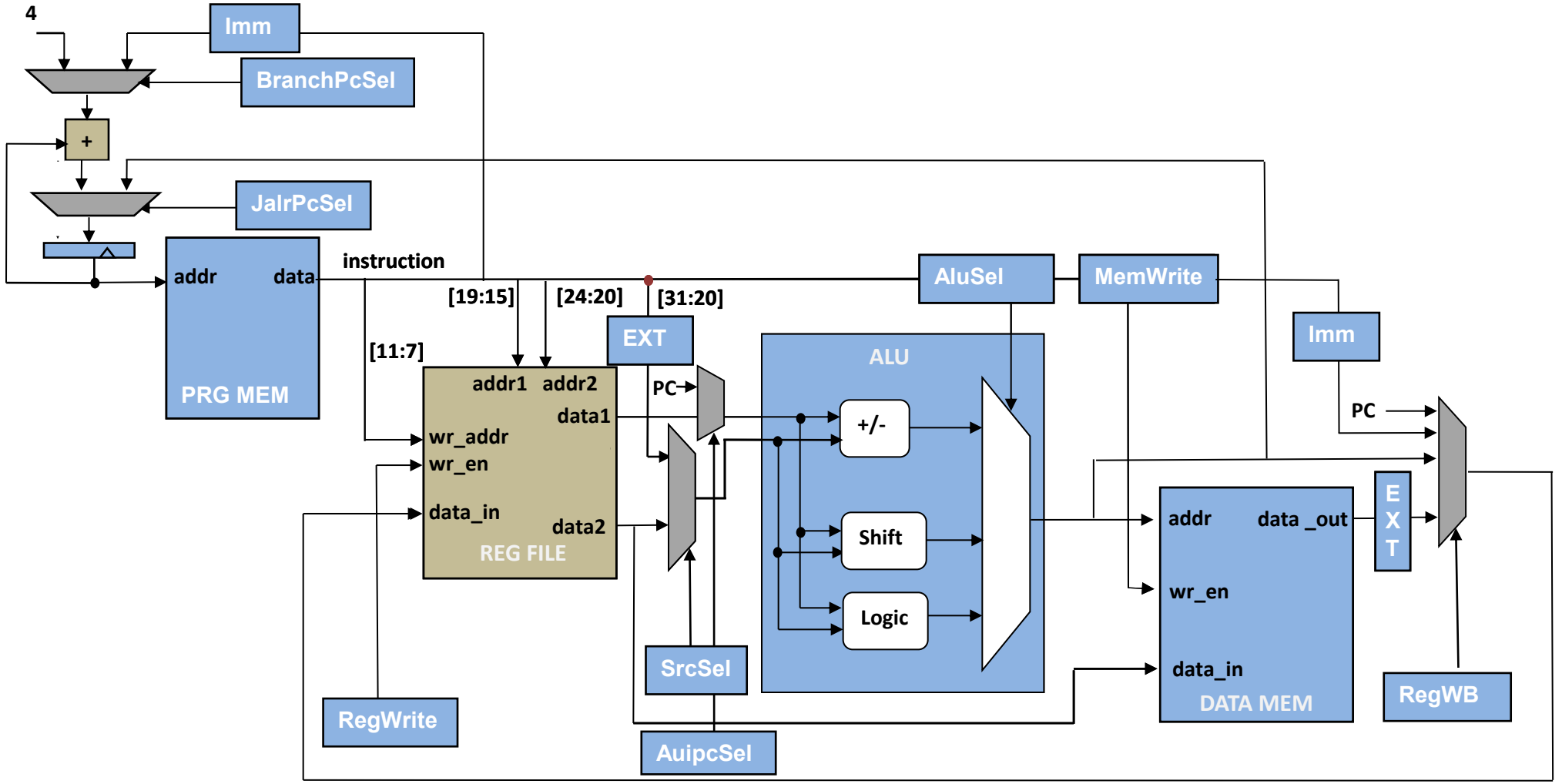
RegWB	
Alu	Alu
Alui	Alu
Lw	Mem
Sw	x
Branch T	x
Branch F	x
Jal	Pc
Jalr	Pc
Auipc	Alu
Lui	Imm



AuipcSel	
Alu	Reg
Alui	Reg
Lw	Reg
Sw	Reg
Branch T	Reg
Branch F	Reg
Jal	x
Jalr	Reg
Auipc	Pc
Lui	x



BranchPcSel	
Alu	Pc+4
Alui	Pc+4
Lw	Pc+4
Sw	Pc+4
Branch T	Pc+imm
Branch F	Pc+4
Jal	Pc+imm
Jalr	x
Auipc	Pc+4
Lui	Pc+4



JalrPcSel	
Alu	Pc+4
Alui	Pc+4
Lw	Pc+4
Sw	Pc+4
Branch T	Pc+4
Branch F	Pc+4
Jal	Pc+4
Jalr	Pc+imm
Auipc	Pc+4
Lui	Pc+4

Control Table

Opcode	Srcsel	Memwrite	Regwrite	RegWB	Auipcsel	Branchsel	Jalrsel
Alu	Reg	No	Yes	Alu	Reg	Pc+4	Pc+4
Alui	Imm	No	Yes	Alu	Reg	Pc+4	Pc+4
Lw	Imm	No	Yes	Mem	Reg	Pc+4	Pc+4
Sw	Imm	Yes	No	x	Reg	Pc+4	Pc+4
Branch T	Reg	No	No	x	Reg	Pc+imm	Pc+4
Branch F	Reg	No	No	x	Reg	Pc+4	Pc+4
Jal	x	No	Yes	Pc	x	Pc+imm	Pc+4
Jalr	Imm	No	Yes	Pc	Reg	x	Pc+imm
Auipc	Imm	No	Yes	Alu	Pc	Pc+4	Pc+4
Lui	x	No	Yes	Imm	x	Pc+4	Pc+4